CASE-STUDY WORKBOOK

Load flow, operating limits and renewable integration with PyPSA

Power System Modeling, Simulation and Control | Sources: Exercise 5 (28 October 2022); Travaux pratiques "Load flow on a real network"; H. Saadat, *Power System Analysis*; PyPSA 1.4

---

1. How every case is solved: the scientific approach

Each exercise is treated in the same eight steps. The steps are the same for a three-bus textbook network and for a full year of real weather: only the data and the questions change.

Method: the eight steps applied to every case

1. **Statement and question**: the exercise as given, with its original diagram; the engineering question it asks.
2. **Data and hypotheses**: every number used, its unit, its source, and every assumption made.
3. **Physical model**: the equations that link the unknowns to the data (load-flow equations, limits, costs).
4. **Solution method**: the algorithm (Newton-Raphson, optimal power flow, N-1 screening, bisection on a limit).
5. **Implementation in PyPSA**: the complete, commented Python code (also in `solutions/`).
6. **Results**: the printed output of that code, tables and figures.
7. **Verification**: an independent check (hand calculation, own solver, energy balance, physical plausibility).
8. **Interpretation and recommendation**: what the numbers mean for the grid, and what to do.

PyPSA in five objects

Table 1: The PyPSA objects used in this workbook.

Object	What it represents	Key attributes
`n = pypsa.Network()`	the grid	`n.snapshots` (time steps)
`n.add("Bus", ...)`	a node	`v_nom` (kV), `v_mag_pu_set` (PV/slack set point)
`n.add("Line", ...)`	a branch (π model)	`r`, `x` (Ω), `b` (S), `s_nom` (MVA)
`n.add("Generator", ...)`	an injection	`control` = Slack / PV / PQ, `p_set`, `q_set`, costs
`n.add("Load", ...)`	a consumption	`p_set` (MW), `q_set` (MVAr)
`n.pf()`	AC load flow (Newton-Raphson)	results in `n.buses_t.v_mag_pu`, `n.lines_t.p0` ...
`n.optimize()`	optimal power flow / investment	results in `n.generators_t.p`, `p_nom_opt`

Link with the per-unit method sheet: from p.u. to the units of PyPSA

PyPSA stores line data in physical units. With the base of the zone of the line (same rule as in the per-unit sheet):

$$Z_B = \frac{(\mathrm{kV}_B)^2}{\mathrm{MVA}_B}, \qquad r_\Omega = r_{pu}\, Z_B, \qquad x_\Omega = x_{pu}\, Z_B, \qquad b_S = \frac{b_{pu}}{Z_B}.$$

Example (Exercise 5): $S_B = 100$ MVA, $V_B = 110$ kV, $Z_B = 121\ \Omega$; $X = 0.125$ p.u. $\Rightarrow 15.125\ \Omega$; $B^0 = 0.1$ p.u. $\Rightarrow 8.26 \times 10^{-4}$ S. The p.u. results do not depend on the choice of V_B.

2. Exercise 5, Assignment 1: Newton's method

Step 1: statement and question

Statement

Perform 5 iterations using Newton's method to solve $f(x) = 10x^2 - 2x^3 = 0$. The answer should include the expression for the derivative $f'(x)$ and the values of x, Δx, $f(x)$ and $f'(x)$ at each iteration. (a) Start at $x = 1$. (b) Start at $x = 4$. (c) Sketch $f(x)$ versus x to see the graphical solutions.

Question behind the exercise: how does the algorithm used by every load-flow program behave, and why does its starting point matter?

Step 2: data and hypotheses

$f(x) = 10x^2 - 2x^3 = 2x^2(5 - x)$: two roots, $x = 0$ (double) and $x = 5$ (simple). No hypothesis is needed.

Step 3: model

$$f'(x) = 20x - 6x^2 = 2x(10 - 3x), \qquad \Delta x_k = -\frac{f(x_k)}{f'(x_k)}, \qquad x_{k+1} = x_k + \Delta x_k.$$

f' vanishes at $x = 0$ and $x = 10/3$: there the tangent is horizontal and no step can be computed.

Step 4: method

Newton replaces f by its tangent at x_k and jumps to the zero of the tangent. Simplified update: $x_{k+1} = x_k \dfrac{10 - 4x_k}{20 - 6x_k}$, which tends to $x_k/2$ near 0.

Step 5: implementation

Code 1: Newton's method, Exercise 5 Assignment 1.

```python
"""Exercise 5, Assignment 1 - Newton's method on f(x) = 10 x^2 - 2 x^3 = 0.

Run from the repository root:  python solutions/s01_newton_method.py
"""
import numpy as np


def f(x):
    return 10 * x**2 - 2 * x**3          # f(x) = 2x^2 (5 - x): roots 0 (double) and 5


def df(x):
    return 20 * x - 6 * x**2             # f'(x) = 2x (10 - 3x): zero at x = 0 and x = 10/3


def newton(x0, n_iter=5):
    """Five Newton steps x_{k+1} = x_k + dx with dx = -f(x_k) / f'(x_k)."""
    x = x0
    print(f"{'k':>2} {'x_k':>12} {'f(x_k)':>12} {'f`(x_k)':>12} {'dx':>12}")
    for k in range(n_iter):
        dx = -f(x) / df(x)
        print(f"{k:>2} {x:12.6f} {f(x):12.6f} {df(x):12.6f} {dx:12.6f}")
        x += dx
    print(f"{n_iter:>2} {x:12.8f}")
    return x


print("(a) start at x0 = 1")
```

```
xa = newton(1.0)
print("\n(b) start at x0 = 4")
xb = newton(4.0)

# Convergence rate: error ratio e_{k+1}/e_k (linear) or e_{k+1}/e_k^2 (quadratic)
print("\nConvergence check")
for x0, root in [(1.0, 0.0), (4.0, 5.0)]:
    x, e = x0, [abs(x0 - root)]
    for _ in range(5):
        x -= f(x) / df(x)
        e.append(abs(x - root))
    e = np.array(e)
    print(f"x0 = {x0}: e_k+1/e_k   = {np.round(e[1:] / e[:-1], 4)}")
    print(f"          e_k+1/e_k^2 = {np.round(e[1:] / e[:-1] ** 2, 4)}")
print("f'(10/3) =", df(10 / 3), "→ no Newton step is possible from x0 = 10/3")
```

Step 6: results

Code 2: Output of Code 1.

```
(a) start at x0 = 1
 k          x_k       f(x_k)      f'(x_k)          dx
 0     1.000000     8.000000    14.000000   -0.571429
 1     0.428571     1.679300     7.469388   -0.224824
 2     0.203747     0.398212     3.825864   -0.104084
 3     0.099663     0.097347     1.933659   -0.050343
 4     0.049319     0.024084     0.971794   -0.024783
 5   0.02453627

(b) start at x0 = 4
 k          x_k       f(x_k)      f'(x_k)          dx
 0     4.000000    32.000000   -16.000000    2.000000
 1     6.000000   -72.000000   -96.000000   -0.750000
 2     5.250000   -13.781250   -60.375000   -0.228261
 3     5.021739    -1.096429   -50.872401   -0.021553
 4     5.000187    -0.009331   -50.007464   -0.000187
 5   5.00000001

Convergence check
x0 = 1.0: e_k+1/e_k   = [0.4286 0.4754 0.4891 0.4949 0.4975]
          e_k+1/e_k^2 = [ 0.4286  1.1093  2.4008  4.9654 10.0872]
x0 = 4.0: e_k+1/e_k   = [1.0e+00 2.5e-01 8.7e-02 8.6e-03 1.0e-04]
          e_k+1/e_k^2 = [1.     0.25   0.3478 0.3948 0.4   ]
f'(10/3) = 0.0 -> no Newton step is possible from x0 = 10/3
```

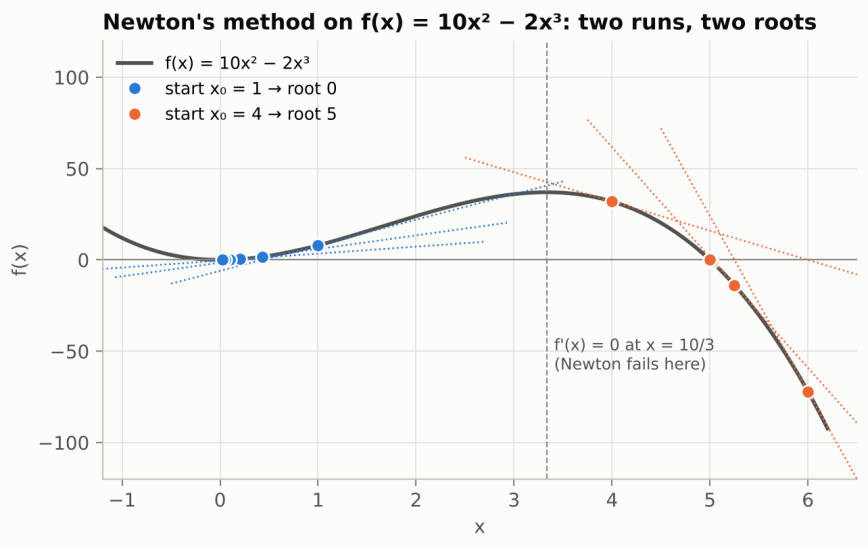


Figure 1: Question (c): $f(x)$, the iterates of both runs and the first tangents.

Step 7: verification

Hand check of the first step from $x_0 = 1$: $f(1) = 8$, $f'(1) = 14$, $\Delta x = -0.5714$, $x_1 = 0.4286$ (same as the output). From $x_0 = 4$: $f(4) = 32$, $f'(4) = -16$, $\Delta x = +2$, $x_1 = 6$. The final values are roots: $f(0.0245) \approx 0.006$, $f(5.00000001) \approx -5 \times 10^{-7}$.

Step 8: interpretation

- From $x_0 = 1$ the method converges to the **double root 0**, but **linearly**: $e_{k+1}/e_k \to 0.5$, after 5 steps $x = 0.0245$.
- From $x_0 = 4$ it overshoots to 6 (negative slope), then converges **quadratically** to 5: $e_{k+1}/e_k^2 \to 0.4$, 8 exact digits after 5 steps.
- At $x_0 = 10/3$, $f' = 0$: the method fails. The root found depends on the starting point.

What I say in front of a jury

"A load flow is a vector version of this method: the Jacobian replaces $f'(x)$. A flat start is a good first guess because grids run near 1 p.u.; quadratic convergence explains why a load flow needs only 3 to 5 iterations; and a singular Jacobian, like $f' = 0$ here, marks the voltage-collapse point."

3. Exercise 5, Assignment 2: Newton-Raphson load flow of a three-bus grid

Step 1: statement and question

Statement

The one-line diagram of a three-phase system with three nodes is shown in Figure 2. Bus 1 is the balance (slack) bus, node 2 a generator bus and node 3 a load bus. (1) Determine the node admittance matrix. (2) Derive the load-flow problem and solve it with the Newton-Raphson method. (3) Determine the power flows and transmission losses.

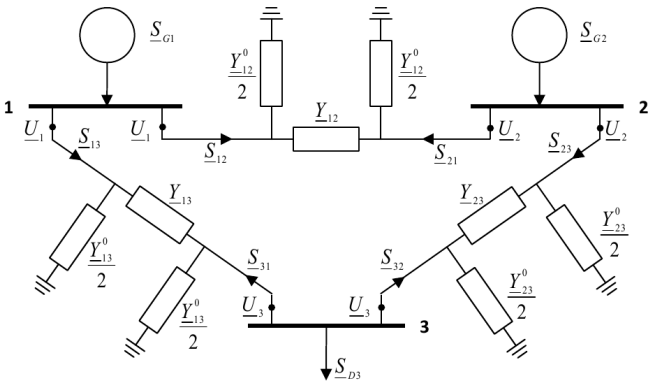


Figure 1: Electrical grid with 3 nodes and reference node

Known:	Unknown:
$\underline{Y}_{ik} = j0{,}1$	θ_2
$\underline{Y}^0_{ik} = -j8$	θ_3
$\theta_1 = 0°$	$\lvert\underline{U}_3\rvert$
$\lvert\underline{U}_1\rvert = 1{,}00$	P_{G1}
$\lvert\underline{U}_2\rvert = 1{,}05$	Q_{G1}
$P_{G2} = 0{,}50$	Q_{G2}
$P_{D3} = 2{,}50$	$\underline{S}_{ik}$
$Q_{D3} = 1{,}00$	$\underline{S}_{ik,v}$
	$\underline{S}_{v,ges}$

Figure 2: Original figure and data table of Exercise 5 (Figure 1: electrical grid with 3 nodes and reference node).

Step 2: data and hypotheses

Table 2: Exercise 5: data used (p.u., $S_B = 100$ MVA).

Bus	Type	Known	Unknown
1	slack	$\lvert U_1\rvert = 1.00, \theta_1 = 0^\circ$	P_{G1}, Q_{G1}
2	PV	$\lvert U_2\rvert = 1.05, P_{G2} = 0.50$	θ_2, Q_{G2}
3	PQ	$P_{D3} = 2.50, Q_{D3} = 1.00$	$\theta_3, \lvert U_3\rvert$
Lines 1-2, 1-3, 2-3	π model	series $\underline{Y}_{ik} = -j8$; shunt $\underline{Y}^0_{ik} = j0.1$ ($j0.05$ per end)	flows, losses

Hypothesis on the line data (to state on the exam paper)

The table gives $\underline{Y}_{ik} = j0.1$ and $\underline{Y}^0_{ik} = -j8$. Taken literally, the series element would be a capacitor and the shunt an 8 p.u. reactor; the problem then has **no solution** (a series admittance of 0.1 p.u. cannot carry 2.5 p.u.; Newton diverges). The physically consistent reading swaps the two values: series admittance $-j8$ (an inductive line, $X = 0.125$ p.u.) and line charging $+j0.1$. Reading both values as impedances ($Z = j0.1$, $Z^0 = -j8$) gives $|U_3| = 0.971$ p.u.: the conclusions are the same.

Step 3: model

Admittance matrix. Each line adds $\underline{y}_{ik} + \underline{Y}^0_{ik}/2$ to Y_{ii} and Y_{kk}, and $-\underline{y}_{ik}$ to Y_{ik}. Each bus has two lines:

$$Y_{ii} = 2(-j8) + 2(j0.05) = -j15.9, \qquad Y_{ik} = +j8, \qquad \mathbf{Y} = \begin{bmatrix} -j15.9 & j8 & j8 \\ j8 & -j15.9 & j8 \\ j8 & j8 & -j15.9 \end{bmatrix}.$$

Load-flow equations (lossless lines, $G = 0$, $B_{ik} = 8$, $B_{ii} = -15.9$):

$$\begin{aligned} P_2 &= 8|U_2||U_1|\sin\theta_{21} + 8|U_2||U_3|\sin\theta_{23} = +0.50 \\ P_3 &= 8|U_3||U_1|\sin\theta_{31} + 8|U_3||U_2|\sin\theta_{32} = -2.50 \\ Q_3 &= 15.9|U_3|^2 - 8|U_3||U_1|\cos\theta_{31} - 8|U_3||U_2|\cos\theta_{32} = -1.00 \end{aligned}$$

Three equations, three unknowns $x = [\theta_2, \theta_3, |U_3|]^T$. There is no Q_2 equation: bus 2 is a PV bus.

Step 4: method (first iteration by hand)

Flat start: $\theta_2 = \theta_3 = 0$, $|U_3| = 1$. Computed injections: $P_2 = P_3 = 0$, $Q_3 = 15.9 - 8 - 8.4 = -0.5$. Mismatches: $\Delta P_2 = 0.5$, $\Delta P_3 = -2.5$, $\Delta Q_3 = -1.0 - (-0.5) = -0.5$. Jacobian at the flat start (e.g. $\partial P_2/\partial\theta_2 = 1.05\,(8+8) = 16.8$, $\partial Q_3/\partial|U_3| = 2\cdot 15.9 - 8 - 8.4 = 15.4$):

$$\begin{bmatrix} 16.8 & -8.4 & 0 \\ -8.4 & 16.4 & 0 \\ 0 & 0 & 15.4 \end{bmatrix} \begin{bmatrix} \Delta\theta_2 \\ \Delta\theta_3 \\ \Delta|U_3| \end{bmatrix} = \begin{bmatrix} 0.5 \\ -2.5 \\ -0.5 \end{bmatrix} \Rightarrow \Delta\theta_2 = -3.578^\circ,\ \Delta\theta_3 = -10.567^\circ,\ \Delta|U_3| = -0.0325.$$

The program repeats this step until the largest mismatch is below 10^{-10}.

Step 5: implementation

Code 3: Newton-Raphson by hand (part A) and with PyPSA (part B).

```
"""Exercise 5, Assignment 2 - Newton-Raphson load flow of the three-bus system.

Part A: the method written by hand (admittance matrix, mismatches, Jacobian, iterations,
```

```
        flows and losses).  Part B: the same network in PyPSA, solved with n.pf().
Data in p.u. on S_base = 100 MVA. Run:  python solutions/s02_ex5_load_flow.py
"""
import numpy as np
import pandas as pd
import pypsa

np.set_printoptions(precision=4, suppress=True)

# ------------------------------------------------------------------ data (p.u.)
y_line = -8j          # series admittance of each line   (X = 0.125 p.u.)
y0_line = 0.1j        # total shunt admittance of each line (y0/2 at each end)
lines = [(0, 1), (0, 2), (1, 2)]                    # lines 1-2, 1-3, 2-3 (0-based)
V = np.array([1.00, 1.05, 1.00])                    # |U1|, |U2| fixed; |U3| flat start
th = np.zeros(3)                                    # theta1 = 0 (reference); flat start
P_spec = np.array([0.0, +0.50, -2.50])              # P_G2 = 0.5 ; P_D3 = 2.5
Q_spec = np.array([0.0, 0.0, -1.00])                # Q_D3 = 1.0
pvpq, pq = [1, 2], [2]                              # unknown angles, unknown magnitudes

# ------------------------------------------------------------- step 1: admittance matrix
Y = np.zeros((3, 3), dtype=complex)
for i, k in lines:
    Y[i, i] += y_line + y0_line / 2
    Y[k, k] += y_line + y0_line / 2
    Y[i, k] -= y_line
    Y[k, i] -= y_line
print("Step 1 - admittance matrix Y (p.u.):\n", Y)


def injections(V, th):
    """S_i = U_i * conj(sum_k Y_ik U_k)  →  P_i + jQ_i."""
    U = V * np.exp(1j * th)
    return U * np.conj(Y @ U)


def jacobian(V, th):
    """Polar Jacobian [[dP/dtheta, dP/dV], [dQ/dtheta, dQ/dV]] for the unknowns."""
    U = V * np.exp(1j * th)
    I = Y @ U
    dS_dth = 1j * np.diag(U) @ np.conj(np.diag(I) - Y @ np.diag(U))
    dS_dV = np.diag(U) @ np.conj(Y @ np.diag(U / V)) + np.conj(np.diag(I)) @ np.diag(U / V)
    return np.block([[dS_dth.real[np.ix_(pvpq, pvpq)], dS_dV.real[np.ix_(pvpq, pq)]],
                     [dS_dth.imag[np.ix_(pq, pvpq)], dS_dV.imag[np.ix_(pq, pq)]]])

# ------------------------------------------------------------- step 2: Newton-Raphson
print("\nStep 2 - Newton-Raphson iterations")
print(f"{'it':>2} {'max|dF|':>10} {'theta2(deg)':>12} {'theta3(deg)':>12} {'|U3|':>9}")
for it in range(10):
    S = injections(V, th)
    dF = np.r_[P_spec[pvpq] - S.real[pvpq], Q_spec[pq] - S.imag[pq]]   # mismatches
    print(f"{it:>2} {abs(dF).max():10.2e} {np.degrees(th[1]):12.4f} {np.degrees(th[2]):12.4f} {V[2]:9.5
     f}")
    if abs(dF).max() < 1e-10:
        break
    J = jacobian(V, th)
    if it == 0:
        print("   Jacobian at the flat start:\n", J)
    dx = np.linalg.solve(J, dF)                     # J [dtheta2, dtheta3, d|U3|] = dF
    th[pvpq] += dx[:2]
    V[pq] += dx[2:]

S = injections(V, th)
print(f"\nResult: |U3| = {V[2]:.4f} p.u., theta2 = {np.degrees(th[1]):.3f} deg, theta3 = {np.degrees(th
     [2]):.3f} deg")
print(f"        P_G1 = {S[0].real:.4f}, Q_G1 = {S[0].imag:.4f}, Q_G2 = {S[1].imag:.4f} p.u.")

# ------------------------------------------------------------- step 3: flows and losses
U = V * np.exp(1j * th)
rows = []
for i, k in lines:
    I_ik = (y_line + y0_line / 2) * U[i] - y_line * U[k]      # current leaving bus i
    I_ki = (y_line + y0_line / 2) * U[k] - y_line * U[i]
    S_ik, S_ki = U[i] * np.conj(I_ik), U[k] * np.conj(I_ki)
```

```
    rows.append({"line": f"{i+1}-{k+1}", "S_ik": np.round(S_ik, 4), "S_ki": np.round(S_ki, 4),
                 "S_ik,v (loss)": np.round(S_ik + S_ki, 4)})
flows = pd.DataFrame(rows).set_index("line")
print("\nStep 3 - line flows and losses (p.u.)\n", flows)
print("Total losses S_v,ges =", np.round(sum(r["S_ik,v (loss)"] for r in rows), 4), "p.u.")

# ------------------------------------------------------------ Part B: PyPSA
S_base, V_nom = 100.0, 110.0              # MVA, kV (any V_nom gives the same p.u. result)
Z_base = V_nom**2 / S_base                # ohm
n = pypsa.Network()
for bus, v_set in [("1", 1.00), ("2", 1.05), ("3", 1.00)]:
    n.add("Bus", bus, v_nom=V_nom, v_mag_pu_set=v_set)
for a, b in [("1", "2"), ("1", "3"), ("2", "3")]:
    z = 1 / y_line                                      # j0.125 p.u.
    n.add("Line", f"L{a}-{b}", bus0=a, bus1=b, r=z.real * Z_base, x=z.imag * Z_base,
          b=y0_line.imag / Z_base, s_nom=500)           # ohm, ohm, siemens
n.add("Generator", "G1", bus="1", control="Slack", p_set=0.0)   # slack: numeric p_set
n.add("Generator", "G2", bus="2", control="PV", p_set=0.50 * S_base)
n.add("Load", "D3", bus="3", p_set=2.50 * S_base, q_set=1.00 * S_base)
n.pf()
print("\nPart B - PyPSA results")
print(pd.DataFrame({"|U| (p.u.)": n.buses_t.v_mag_pu.iloc[0],
                    "theta (deg)": np.degrees(n.buses_t.v_ang.iloc[0])}).round(5))
print((n.generators_t.p.iloc[0] / S_base).round(5).rename("P (p.u.)").to_frame()
      .join((n.generators_t.q.iloc[0] / S_base).round(5).rename("Q (p.u.)")))
print("max |U3 own - U3 PyPSA| =", abs(V[2] - n.buses_t.v_mag_pu.iloc[0]["3"]))
```

Step 6: results

Code 4: Output of Code 2.

```
Step 1 - admittance matrix Y (p.u.):
 [[0.-15.9j 0. +8.j  0. +8.j ]
 [0. +8.j  0.-15.9j 0. +8.j ]
 [0. +8.j  0. +8.j  0.-15.9j]]

Step 2 - Newton-Raphson iterations
it    max|dF|  theta2(deg)  theta3(deg)      |U3|
 0   2.50e+00       0.0000       0.0000   1.00000
   Jacobian at the flat start:
 [[16.8 -8.4  0. ]
 [-8.4 16.4  0. ]
 [ 0.   0.  15.4]]
 1   2.08e-01      -3.5782     -10.5668   0.96753
 2   4.13e-03      -3.6522     -11.0830   0.95182
 3   1.85e-06      -3.6530     -11.0949   0.95149
 4   3.91e-13      -3.6530     -11.0949   0.95149

Result: |U3| = 0.9515 p.u., theta2 = -3.653 deg, theta3 = -11.095 deg
        P_G1 = 2.0000, Q_G1 = 0.0474, Q_G2 = 1.2216 p.u.

Step 3 - line flows and losses (p.u.)
                 S_ik            S_ki   S_ik,v (loss)
line
1-2   0.5352-0.4329j -0.5352+0.3819j  0.0000-0.0510j
1-3   1.4648+0.4803j -1.4648-0.2722j  0.0000+0.2081j
2-3   1.0352+0.8397j -1.0352-0.7278j  0.0000+0.1119j
Total losses S_v,ges = 0.269j p.u.

Part B - PyPSA results
      |U| (p.u.)  theta (deg)
name
1        1.00000      0.00000
2        1.05000     -3.65302
3        0.95149    -11.09490
      P (p.u.)  Q (p.u.)
name
G1         2.0   0.04739
G2         0.5   1.22160
max |U3 own - U3 PyPSA| = 1.1102230246251565e-16
```

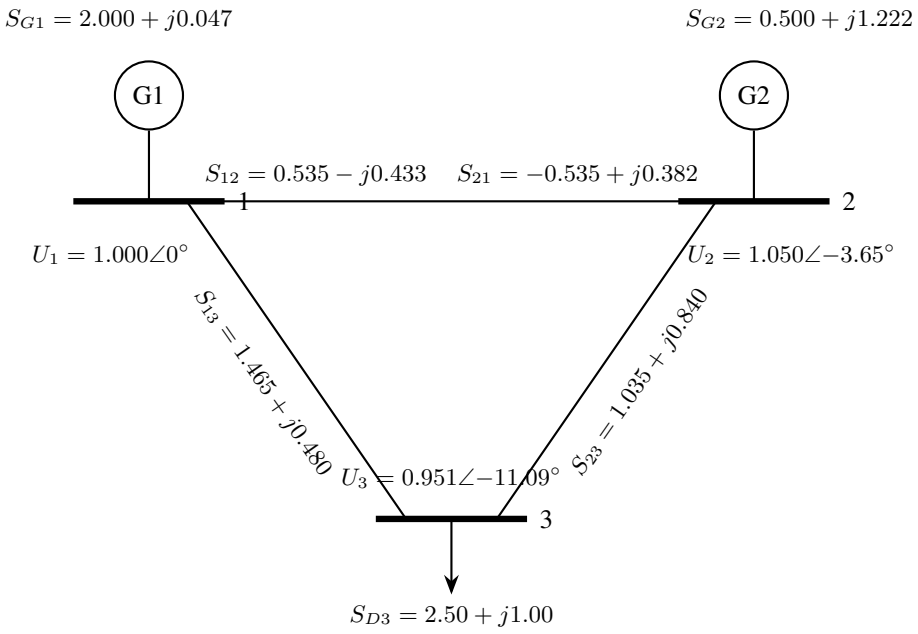


Figure 3: Solved Exercise 5 (p.u.): voltages, generation and sending-end flows.

Table 3: Exercise 5: line flows and losses (p.u.).

Line	S_{ik}	S_{ki}	$S_{ik,v} = S_{ik} + S_{ki}$
1-2	$0.5352 - j0.4329$	$-0.5352 + j0.3819$	$0 - j0.0510$
1-3	$1.4648 + j0.4803$	$-1.4648 - j0.2722$	$0 + j0.2081$
2-3	$1.0352 + j0.8397$	$-1.0352 - j0.7278$	$0 + j0.1119$
Total			$S_{v,ges} = 0 + j0.2690$

Step 7: verification

- **Active balance:** $P_{G1} + P_{G2} - P_{D3} = 2.000 + 0.500 - 2.500 = 0$ = active losses (lossless lines). ✓
- **Reactive balance:** $Q_{G1} + Q_{G2} - Q_{D3} = 0.047 + 1.222 - 1.000 = 0.269 = Q_{v,ges}$. ✓
- **Independent solver:** PyPSA gives the same voltages and powers to 10^{-15}. ✓
- **Convergence:** mismatch $2.5 \to 0.21 \to 4.1 \times 10^{-3} \to 1.9 \times 10^{-6} \to 3.9 \times 10^{-13}$: quadratic, as expected.

Step 8: interpretation

The load bus sits at 0.951 p.u., just above the usual lower limit. Line 1-3 carries most of the load (1.46 p.u.). Bus 2, held at 1.05 p.u., exports reactive power (1.22 p.u.) towards bus 3 and even towards bus 1. Line 1-2 is lightly loaded: its charging exceeds its series losses, so its reactive "loss" is negative ($-j0.051$).

What I say in front of a jury

"I built $\mathbf{Y}$ line by line, wrote three equations for three unknowns, and solved them with Newton-Raphson in 4 iterations. The slack supplies exactly 2.0 p.u. because the lines are lossless; all losses are reactive (0.269 p.u.). PyPSA reproduces my result to machine precision, which validates both the model and the data conversion."

4. Travaux pratiques: load flow on a real network (common thread)

Statement (common thread of the practical work)

We follow a case inspired by Niger: *integration of a solar PV plant into a grid supplying Niamey*. Central question: if demand grows and a solar plant is connected, do voltages stay acceptable? Can the lines carry the power? Where do losses and congestion appear? The networks and numbers are **educational and fictitious**; they do not represent the real Niamey grid. We go from 3 to 5, 9 and 14 buses.

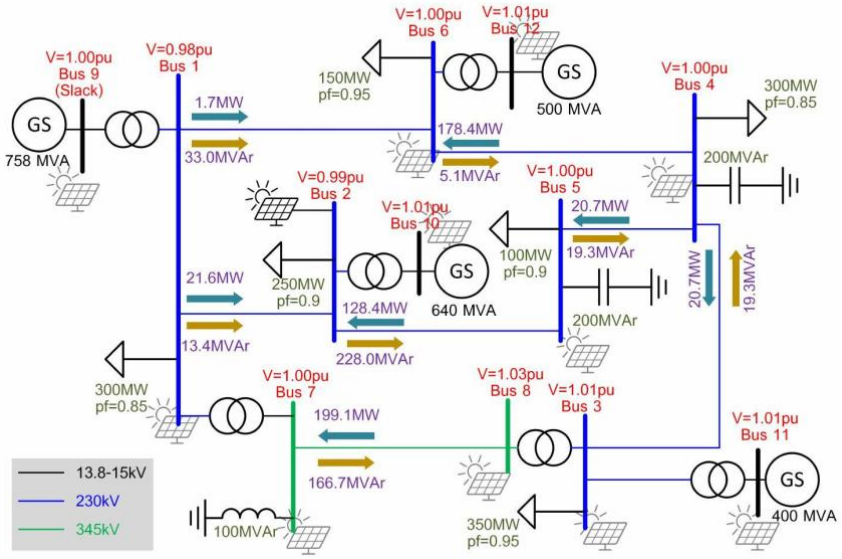


Figure 4: Original illustration of the practical work (multi-voltage grid with solar plants).

Simulation protocol of the practical work (applied in every TP case)

1. Build the network (buses, lines, transformers, generators, loads). 2. Define the electrical parameters (nominal voltage, R, X, thermal limits, P, Q). 3. Define the bus types (slack, PV, PQ). 4. Run the AC load flow. 5. Modify the scenario (more demand, solar, set points, battery). 6. Compare, interpret, recommend.

Indicators always reported: V_i (p.u.), θ_i (°), P_{ij} (MW), Q_{ij} (MVAr), loading (%), P_{loss} (MW).

Table 4: Line data used in the Niamey-context cases (132 kV overhead line, ACSR 240/40 mm^2).

Parameter	Value	Note
R, X	0.12 and 0.39 Ω/km	typical for 132 kV ACSR
B (charging)	2.9 μS/km	
Thermal rating	100 MVA	hot-season value (45 °C ambient), assumption
Base	$V_B = 132$ kV, $Z_B = 132^2/100 = 174.24\ \Omega$	so 25 km: $X = 9.75\ \Omega = 0.056$ p.u.

5. TP case 1: three-bus network, power sharing

Step 1: statement and question

Statement

Bus 1	conventional power plant	slack bus
Bus 2	city / load	50 MW + 20 MVAr
Bus 3	solar power plant	30 MW

The three buses are interconnected by lines. First approximation, without losses: $P_{slack} = 50 - 30 = 20$ MW. Analyse: (1) are the voltages at buses 2 and 3 close to nominal? (2) Does the solar plant reduce the conventional output? (3) Is there enough reactive power? (4) Are the lines adequately sized? (5) What are the total active losses?

Q2. (40) Fig. 1 shows the one-line diagram of a simple three-bus power system with generation at bus 1. The magnitude of voltage at bus 1 is adjusted to 1.05 pu. The scheduled loads at buses 2 and 3 are as marked on the diagram. Line impedances are marked in pu on a 100 MVA base and the line charging susceptances are neglected.

a) (30) Using the GS (Gauss-Seidael) method, voltage phasors at the load buses 2 and 3 (P-Q buses) are to be calculated. Starting the initial guess with $V_2^{(0)} = 1.0 + j0.0\ pu$ and $V_3^{(0)} = 1.0 + j0.0\ pu$, calculate $V_2^{(1)}$ and $V_3^{(1)}$ accurate to 4 decimal places. (*Perform one iteration*).

b) (10) Using $V_2^{(1)}$ and $V_3^{(1)}$ you have calculated in (b), calculated the power flow $S_{12}^{(1)}$ flowing from bus1 to bus2.

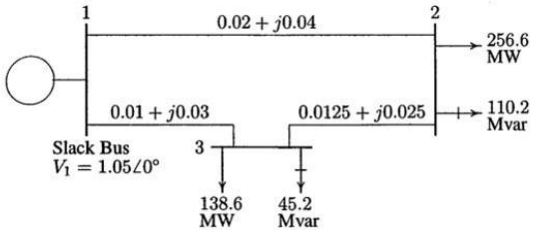


Figure 5: Original three-bus figure of the practical work (illustration of a three-bus load-flow problem).

Step 2: data and hypotheses

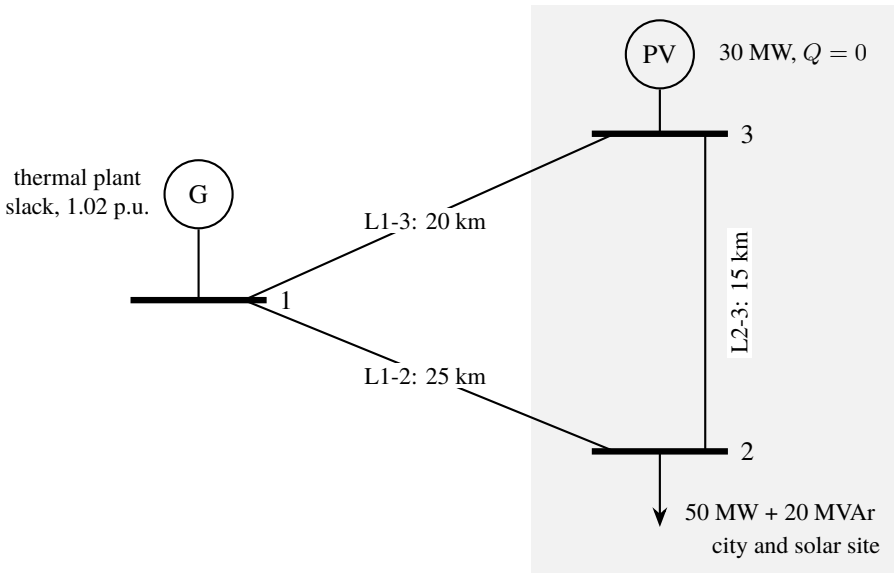


Figure 6: Network modelled in PyPSA (132 kV, ratings 100 MVA).

Hypotheses: steady state, balanced three-phase, PV plant at unity power factor (PQ) unless stated, slack

set point 1.02 p.u. For question (6) of our extension, the real solar profile of 4 May 2023 (NASA POWER, Niamey, the day of the annual demand peak) and a modelled demand shape are used.

Step 3: model

AC load-flow equations of section 3 for every bus; line impedances $Z = (0.12 + j0.39)\,\ell\ \Omega$; slack balance $P_{slack} = \sum P_{load} - P_{PV} + P_{loss}$; voltage drop $\Delta V \approx (RP + XQ)/V$.

Step 4: method

Three variants at the design point (no PV; PV at unity power factor; PV in voltage-control mode at 1.02 p.u.), each solved with `n.pf()`; then 24 load flows for the real day.

Step 5: implementation

Code 5: TP case 1 in PyPSA.

```python
"""TP case 1 - Three-bus network (Niamey context): thermal plant, city, solar plant.

Questions: voltages, effect of solar on the thermal output, reactive power, line loading,
losses. Part A: design point built explicitly in PyPSA. Part B: the real day of 4 May 2023
(24 snapshots, real solar profile) from the dataset data/networks/niamey_3bus.
Run from the repository root:  python solutions/s03_tp1_power_sharing_3bus.py
"""
import logging
import warnings

import numpy as np
import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)

# 132 kV overhead line, ACSR 240/40 mm2 (per-km values) and hot-season rating
R_KM, X_KM, B_KM, S_NOM = 0.12, 0.39, 2.9e-6, 100.0     # ohm/km, ohm/km, S/km, MVA


def network(pv_mw, pv_mode="PQ"):
    n = pypsa.Network()
    n.add("Bus", "1", v_nom=132, v_mag_pu_set=1.02)        # thermal plant (slack)
    n.add("Bus", "2", v_nom=132)                           # city substation
    n.add("Bus", "3", v_nom=132, v_mag_pu_set=1.02)        # solar plant
    for name, a, b, km in [("L1-2", "1", "2", 25), ("L1-3", "1", "3", 20), ("L2-3", "2", "3", 15)]:
        n.add("Line", name, bus0=a, bus1=b, r=R_KM * km, x=X_KM * km, b=B_KM * km, s_nom=S_NOM)
    n.add("Generator", "thermal", bus="1", control="Slack", p_set=0.0)
    n.add("Generator", "pv", bus="3", control=pv_mode, p_set=pv_mw, q_set=0.0)
    n.add("Load", "city", bus="2", p_set=50.0, q_set=20.0)
    n.pf()
    return n


def report(n):
    flows = n.lines_t
    s = np.maximum(np.hypot(flows.p0, flows.q0), np.hypot(flows.p1, flows.q1)).iloc[0]
    return {"|V2|": n.buses_t.v_mag_pu.iloc[0]["2"], "|V3|": n.buses_t.v_mag_pu.iloc[0]["3"],
            "P_th MW": n.generators_t.p.iloc[0]["thermal"],
            "Q_th MVAr": n.generators_t.q.iloc[0]["thermal"],
            "Q_pv MVAr": n.generators_t.q.iloc[0]["pv"],
            "max load %": 100 * (s / n.lines.s_nom).max(),
            "loss MW": (flows.p0 + flows.p1).iloc[0].sum()}

# ---------------------------------------------------------------- Part A: design point
cases = {"no PV": network(0.0), "PV 30, unity PF": network(30.0),
         "PV 30, voltage ctrl": network(30.0, "PV")}
print("Part A - design point (city 50 MW + 20 MVAr, PV at 30 MW)")
print(pd.DataFrame({k: report(n) for k, n in cases.items()}).T.round(3).to_string())
n = cases["PV 30, unity PF"]
print("\nLine flows with 30 MW of PV (MW, MVAr):")
```

```
print(pd.DataFrame({"P0": n.lines_t.p0.iloc[0], "Q0": n.lines_t.q0.iloc[0],
                    "P1": n.lines_t.p1.iloc[0], "loss": n.lines_t.p0.iloc[0] + n.lines_t.p1.iloc[0]}).
    round(3))
print("Inverter capability at full output (33 MVA rating): Q_max = %.1f MVAr" % np.sqrt(33**2 - 30**2))

# ------------------------------------------------------------------ Part B: the real day
d = pypsa.Network("data/networks/niamey_3bus")         # same network + 24 h time series
d.pf()                                                 # one AC load flow per hour
day = pd.DataFrame({"demand": d.loads_t.p["city"], "PV": d.generators_t.p["pv"],
                    "thermal": d.generators_t.p["thermal"],
                    "losses": (d.lines_t.p0 + d.lines_t.p1).sum(axis=1),
                    "|V2|": d.buses_t.v_mag_pu["2"]})
day.index = day.index.hour
print("\nPart B - 4 May 2023, hourly (MW, p.u.)")
print(day.iloc[::3].round(2).to_string())
e = day[["demand", "PV", "thermal", "losses"]].sum()
print("Daily energy (MWh):", e.round(1).to_dict(), "| solar share %.1f %%" % (100 * e.PV / e.demand))
```

Step 6: results

Code 6: Output of Code 3.

```
Part A - design point (city 50 MW + 20 MVAr, PV at 30 MW)
                     |V2|   |V3|  P_th MW  Q_th MVAr  Q_pv MVAr  max load %  loss MW
no PV                1.009  1.014   50.280     17.790      0.000      31.317    0.280
PV 30, unity PF      1.011  1.017   20.192     17.496      0.000      31.930    0.192
PV 30, voltage ctrl  1.013  1.020   20.180      5.733     11.713      33.196    0.180

Line flows with 30 MW of PV (MW, MVAr):
          P0      Q0      P1   loss
name
L1-2  19.304  10.463 -19.222  0.082
L1-3   0.888   7.033  -0.880  0.008
L2-3 -30.778  -8.501  30.880  0.102
Inverter capability at full output (33 MVA rating): Q_max = 13.7 MVAr

Part B - 4 May 2023, hourly (MW, p.u.)
          demand     PV  thermal  losses  |V2|
snapshot
0          34.55   0.00    34.69    0.13  1.01
3          30.18   0.00    30.28    0.10  1.01
6          32.72   0.33    32.51    0.12  1.01
9          41.21  13.00    28.35    0.14  1.01
12         45.83  20.03    25.97    0.17  1.01
15         45.97  13.85    32.31    0.18  1.01
18         48.05   1.16    47.15    0.25  1.01
21         48.34   0.00    48.61    0.26  1.01
Daily energy (MWh): {'demand': 980.3, 'PV': 146.9, 'thermal': 837.4, 'losses': 4.1} | solar share 15.0 %
```

Step 7: verification

Active balance with PV: $P_{slack} + P_{PV} = 20.19 + 30 = 50.19$ MW $= 50 + 0.19$ (losses). ✓ Daily energy: $837.4 + 146.9 = 984.3$ MWh $= 980.3 + 4.1$ (losses). ✓ Order of magnitude: on 25 km, $\Delta V \approx (3.0 \cdot 19.3 + 9.75 \cdot 10.5)/132^2 \approx 0.9\,\%$, consistent with $1.020 \to 1.011$ p.u. ✓

Step 8: interpretation

1. Voltages: 1.009-1.017 p.u., close to nominal.
2. Solar reduces the thermal output from 50.3 to 20.2 MW (estimate $50 - 30 = 20$ MW, the rest is losses).
3. Reactive power: at unity PF the thermal plant gives 17.5 MVAr; in voltage control the inverters give 11.7 MVAr, below their 13.7 MVAr capability (33 MVA inverters at full output).
4. Lines: at most 33 % of the rating: adequate.
5. Losses: 0.28 MW without PV, 0.19 MW with PV (generation closer to the load).

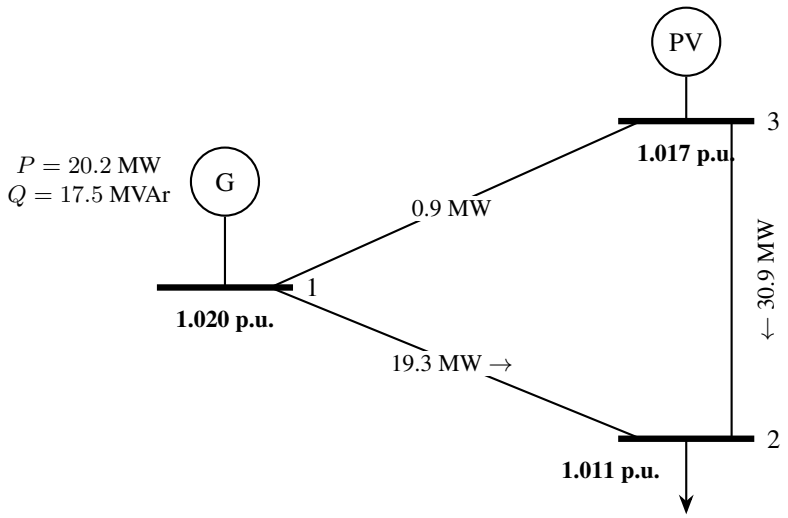


Figure 7: Solved design point with 30 MW of PV at unity power factor: voltages and active flows.

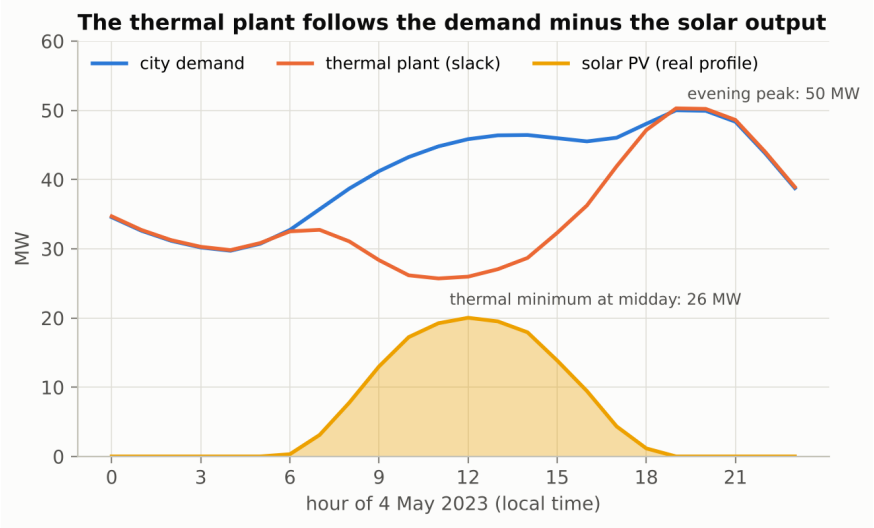


Figure 8: Real day (4 May 2023): the thermal plant follows the demand minus the solar output.

On the real peak day the PV plant gives at most 20 MW and 15 % of the energy; the peak is at 19:00, after sunset.

What I say in front of a jury

"In this first case I study a three-bus grid to understand how the load is shared between conventional and solar generation. The load flow gives the nodal voltages, the flows and the losses. Solar cuts the thermal output by 30 MW at the design point and reduces the losses, but the evening peak stays entirely on the thermal plant."

6. TP case 2: five-bus network, undervoltage

Step 1: statement and question

Statement

Bus 1	conventional generator	slack
Bus 2	substation	load 25 MW
Bus 3	solar power plant	40 MW
Bus 4	urban area	45 MW + 15 MVAr
Bus 5	industrial area	35 MW + 18 MVAr

Bus 5 is far from generation, connected by a relatively long line. The statement *assumes* the result $V_1 = 1.00$, $V_2 = 0.99$, $V_3 = 1.01$, $V_4 = 0.97$, $V_5 = 0.92$ p.u. (illustrative values, not a simulation) and asks which problem is solved (undervoltage, limit 0.95 p.u.) and which solutions an engineer can study.

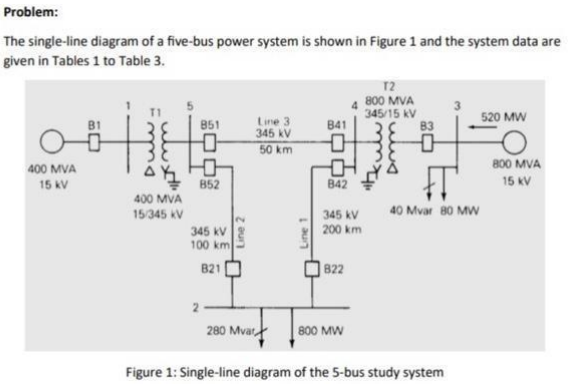


Figure 9: Original five-bus figure of the practical work.

Step 2: data and hypotheses

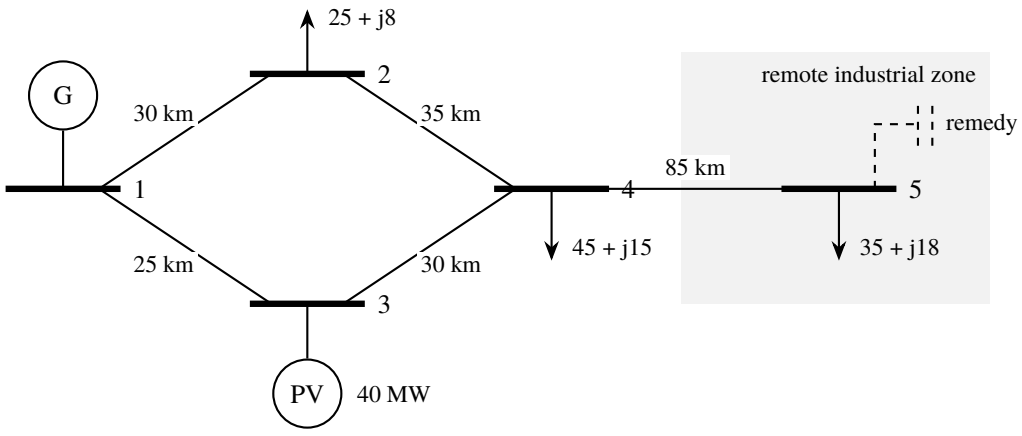


Figure 10: Network modelled in PyPSA (132 kV). Loads in MW + jMVAr; dashed: capacitor bank studied as remedy.

Hypotheses: same line data as case 1; slack 1.02 p.u.; the statement leaves the length free: 85 km is chosen so that the *computed* V_5 matches the order of magnitude of the statement (0.92 p.u.); substation load 25 MW + 8 MVAr (power factor 0.95, assumption). Two design points: A = statement (PV at 40 MW); B = evening peak, no sun.

Step 3: model

$$\Delta V_{4\to5} \approx \frac{RP + XQ}{V}, \qquad R = 0.12 \cdot 85 = 10.2\,\Omega, \quad X = 0.39 \cdot 85 = 33.2\,\Omega.$$

$X \approx 3R$: each MVAr transported costs three times more voltage than each MW. A capacitor bank Q_c at bus 5 produces $Q_c V^2$ locally and removes that reactive flow from the line.

Step 4: method

(1) AC load flow for A and B. (2) Five remedies at the worst case. (3) Smallest capacitor by bisection on $V_5 \geq 0.95$. (4) The real day with and without the bank.

Step 5: implementation

Code 7: TP case 2 in PyPSA.

```python
"""TP case 2 - Five-bus network: undervoltage at a remote industrial zone, and its remedies.

Run from the repository root:  python solutions/s04_tp2_undervoltage_5bus.py
"""
import logging
import warnings

import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
R_KM, X_KM, B_KM = 0.12, 0.39, 2.9e-6                  # 132 kV ACSR 240/40, per km
LINES = [("L1-2", "1", "2", 30), ("L1-3", "1", "3", 25), ("L2-4", "2", "4", 35),
         ("L3-4", "3", "4", 30), ("L4-5", "4", "5", 85)]          # the 85 km feeder to bus 5
LOADS = [("substation", "2", 25, 8), ("urban", "4", 45, 15), ("industry", "5", 35, 18)]


def network(pv_mw=0.0, remedy=None):
    n = pypsa.Network()
    for b in "12345":
        n.add("Bus", b, v_nom=132, v_mag_pu_set=1.02 if b == "1" else 1.0)
    for name, a, b, km in LINES:
        n.add("Line", name, bus0=a, bus1=b, r=R_KM * km, x=X_KM * km, b=B_KM * km, s_nom=100)
    n.add("Generator", "thermal", bus="1", control="Slack", p_set=0.0)
    n.add("Generator", "pv", bus="3", control="PQ", p_set=pv_mw, q_set=0.0)
    for name, bus, p, q in LOADS:
        n.add("Load", name, bus=bus, p_set=p, q_set=q)
    if remedy:
        remedy(n)
    n.pf()
    return n


def v(n):
    return n.buses_t.v_mag_pu.iloc[0]


def losses(n):
    return (n.lines_t.p0 + n.lines_t.p1).iloc[0].sum()

# ------------------------------------------------------------ 1. voltage profile
A, B = network(pv_mw=40.0), network(pv_mw=0.0)
print("1. Voltage profile (p.u.)")
print(pd.DataFrame({"A: PV 40 MW": v(A), "B: evening peak, no PV": v(B)}).round(4).T.to_string())
print("losses A %.2f MW, B %.2f MW" % (losses(A), losses(B)))

# ------------------------------------------------------------ 2. remedies (worst case B)
def capacitor(q):                          # shunt capacitor bank of q MVAr at 1 p.u.
    return lambda n: n.add("ShuntImpedance", "cap5", bus="5", b=q / 132**2)


def setpoint(n):                           # raise the plant voltage set point
```

```
    n.buses.loc["1", "v_mag_pu_set"] = 1.05


def second_circuit(n):                  # double the 4-5 line: impedance / 2, charging x 2
    n.lines.loc["L4-5", ["r", "x"]] /= 2
    n.lines.loc["L4-5", "b"] *= 2


def pv_q_at_night(n):                   # PV inverters as a STATCOM holding 1.03 p.u. at bus 3
    n.generators.loc["pv", "control"] = "PV"
    n.buses.loc["3", "v_mag_pu_set"] = 1.03


def svc(n):                             # SVC/STATCOM at bus 5 holding 0.97 p.u.
    n.add("Generator", "svc5", bus="5", control="PV", p_set=0.0)
    n.buses.loc["5", "v_mag_pu_set"] = 0.97

rows = {}
for name, fn in [("none", None), ("capacitor 15 MVAr", capacitor(15)), ("set point 1.05", setpoint),
                 ("second circuit", second_circuit), ("PV Q-at-night", pv_q_at_night), ("SVC at bus 5",
      svc)]:
    n = network(0.0, fn)
    q_dev = n.generators_t.q.iloc[0].get("svc5", n.generators_t.q.iloc[0]["pv"])
    rows[name] = {"V5 (p.u.)": v(n)["5"], "V1 (p.u.)": v(n)["1"], "losses (MW)": losses(n),
                  "Q of device (MVAr)": q_dev if name in ("PV Q-at-night", "SVC at bus 5") else None}
print("\n2. Remedies at the evening peak")
print(pd.DataFrame(rows).T.round(4).to_string())

# ------------------------------------------------------------ 3. size the capacitor bank
lo, hi = 0.0, 30.0                      # bisection: smallest Q giving V5 >= 0.95 p.u.
for _ in range(30):
    mid = (lo + hi) / 2
    lo, hi = (mid, hi) if v(network(0.0, capacitor(mid)))["5"] < 0.95 else (lo, mid)
print(f"\n3. Smallest capacitor bank: {hi:.1f} MVAr → choose 15 MVAr (3 x 5 MVAr steps)")

# ------------------------------------------------------------ 4. the real day, with the bank
d0 = pypsa.Network("data/networks/niamey_5bus"); d0.pf()
d1 = pypsa.Network("data/networks/niamey_5bus")
d1.add("ShuntImpedance", "cap5", bus="5", b=15 / 132**2); d1.pf()
day = pd.DataFrame({"V5 no bank": d0.buses_t.v_mag_pu["5"], "V5 15 MVAr": d1.buses_t.v_mag_pu["5"]})
day.index = day.index.hour
print("\n4. Real day 4 May 2023 (every 3 h)\n", day.iloc[::3].round(3).T.to_string())
print("hours < 0.95 p.u.: no bank %d, with bank %d" % ((day.iloc[:, 0] < 0.95).sum(), (day.iloc[:, 1] <
      0.95).sum()))
```

Step 6: results

Code 8: Output of Code 4.

```
1. Voltage profile (p.u.)
name                       1       2       3       4       5
A: PV 40 MW             1.02  0.9965  1.0060  0.9806  0.9231
B: evening peak, no PV  1.02  0.9944  1.0003  0.9765  0.9187
losses A 2.68 MW, B 3.18 MW

2. Remedies at the evening peak
                  V5 (p.u.)  V1 (p.u.)  losses (MW)  Q of device (MVAr)
none                 0.9187       1.02       3.1771                 NaN
capacitor 15 MVAr    0.9588       1.02       2.7184                 NaN
set point 1.05       0.9532       1.05       2.9559                 NaN
second circuit       0.9557       1.02       2.4839                 NaN
PV Q-at-night        0.9415       1.02       3.2169             68.1189
SVC at bus 5         0.9700       1.02       2.6599             17.8977

3. Smallest capacitor bank: 11.8 MVAr -> choose 15 MVAr (3 x 5 MVAr steps)

4. Real day 4 May 2023 (every 3 h)
 snapshot       0      3      6      9      12     15     18    21
V5 no bank  0.932  0.936  0.934  0.928  0.925  0.924  0.921  0.92
V5 15 MVAr  0.973  0.976  0.974  0.969  0.966  0.965  0.961  0.96
hours < 0.95 p.u.: no bank 24, with bank 0
```

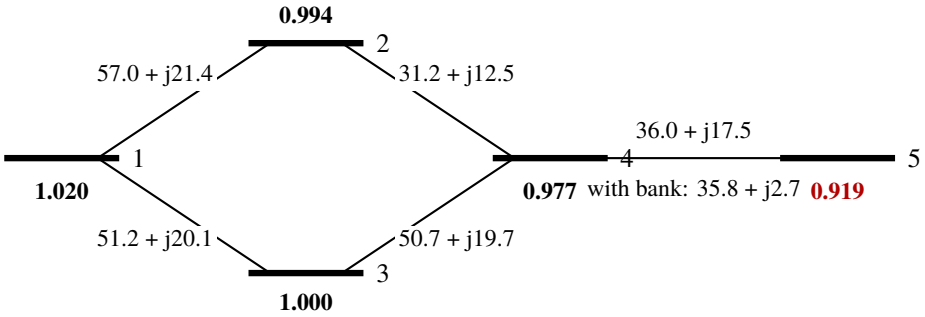


Figure 11: Solved worst case B (evening peak, no PV): voltages (p.u.) and sending-end flows (MW + jMVAr). With the 15 MVAr bank, the reactive flow on line 4-5 falls from 17.5 to 2.7 MVAr and V_5 rises to 0.959 p.u.

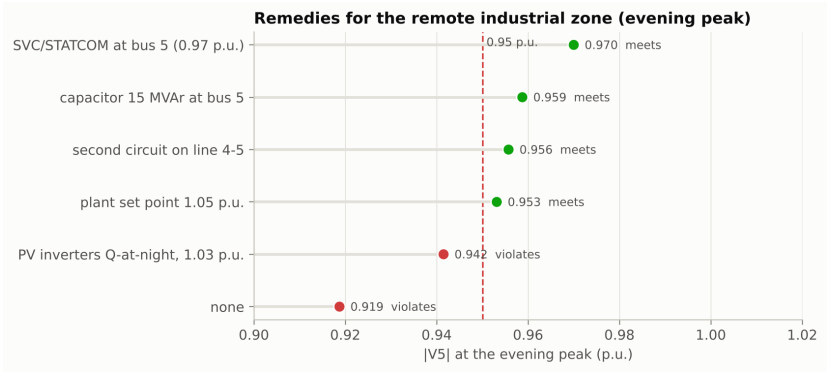


Figure 12: The five remedies at the evening peak.

Step 7: verification

Hand estimate on line 4-5 (case B): $\Delta V \approx (10.2 \cdot 36.0 + 33.2 \cdot 17.5)/132^2 = 5.4\,\%$; computed drop $0.977 - 0.919 = 5.8\,\%$ (the difference is the I^2X term and the voltage level). ✓ Sensitivity: about 3.6 MVAr per percent of voltage at bus 5, so 11.8 MVAr for the missing 3.1 points. ✓

Step 8: interpretation

Only bus 5 is out of limits, 24 hours a day: the problem is **structural** (distance and reactive demand), not a peak accident. The PV plant does not help (far away, no sun at the peak; its inverters would need 68 MVAr, more than their 44 MVA). **Recommendation:** a 15 MVAr switched capacitor bank (3×5 MVAr) at bus 5; cheapest, it also cuts losses by 14 % at the peak; keep a second circuit as the long-term option.

What I say in front of a jury

"In the five-bus network the problem is localised: bus 5 falls to 0.92 p.u. because it is fed through a long line and its load is strongly reactive. I compared five remedies with the load flow; a 15 MVAr capacitor bank restores 0.96 p.u. all day and reduces the losses."

7. TP case 3: IEEE 9-bus system, congestion

Step 1: statement and question

Statement

Reference IEEE 9-bus system: several generators, several loads, interconnected lines. New problem: line congestion. *Suppose a line has a thermal limit of 100 MVA and carries 115 MVA after the load flow: loading 115 %* (illustrative). Solutions to study: redispatch of generation, line reinforcement, topology change, new link, storage.

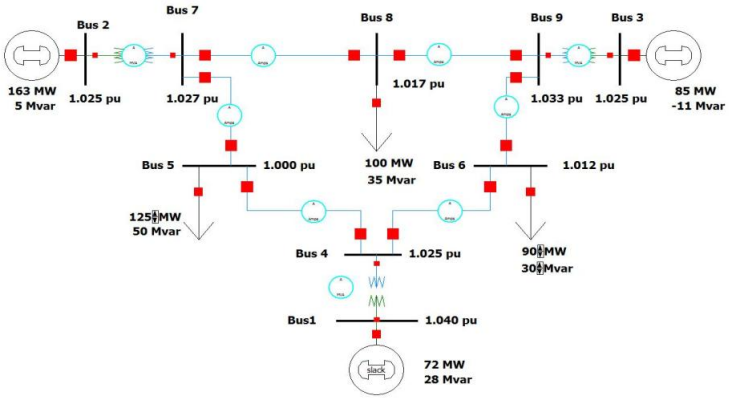


Figure 13: Original IEEE 9-bus figure of the practical work.

Step 2: data and hypotheses

Official MATPOWER data `case9.m` (written out in Code 5): 345 kV, 100 MVA base, ratings 150-300 MVA, quadratic generator costs. To create the congestion with a physical cause (instead of an arbitrary 100 MVA): hot-season ratings 15 % below the MATPOWER values, and an evening peak 15 % above the base load. Security criterion N-1: every single line outage must leave all other lines within their rating.

Step 3: model

Economic dispatch (DC optimal power flow): $\min \sum_g (c_{2,g} P_g^2 + c_{1,g} P_g)$ s.t. nodal balance and $|P_\ell| \leq s_{nom,\ell}$. Security-constrained OPF: the same with the post-outage flows $P_\ell + \text{LODF}_{\ell c}\, P_c \leq s_{nom,\ell}$ for every outage c. The locational marginal price (LMP) is the dual variable of the nodal balance.

Step 4: method

(1) Economic dispatch. (2) AC check of the intact grid. (3) AC N-1 screening. (4) Security-constrained OPF and new AC screening. (5) Reactive support sized by bisection for the voltage problem found. (6) Reinforcement option.

Step 5: implementation

Code 9: TP case 3 in PyPSA.

```
"""TP case 3 - IEEE 9-bus system: congestion, N-1 security, redispatch and nodal prices.

Official data of MATPOWER case9.m (100 MVA base, 345 kV), written out below.
Scenario: hot-season ratings (-15 %) and evening peak (+15 % demand).
Run from the repository root:  python solutions/s05_tp3_congestion_ieee9.py
"""
```

```
import logging
import warnings

import numpy as np
import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
S_B, V_N = 100.0, 345.0
Z_B = V_N**2 / S_B
# MATPOWER case9 branch data: from, to, r, x, b (p.u.), rateA (MVA)
BRANCH = [(1, 4, 0.0, 0.0576, 0.0, 250), (4, 5, 0.017, 0.092, 0.158, 250),
          (5, 6, 0.039, 0.17, 0.358, 150), (3, 6, 0.0, 0.0586, 0.0, 300),
          (6, 7, 0.0119, 0.1008, 0.209, 150), (7, 8, 0.0085, 0.072, 0.149, 250),
          (8, 2, 0.0, 0.0625, 0.0, 250), (8, 9, 0.032, 0.161, 0.306, 250),
          (9, 4, 0.01, 0.085, 0.176, 250)]
# generators: bus, P (MW), V set point, cost c2 ($/MW^2h), c1 ($/MWh), c0 ($/h), Pmax
GEN = [(1, 72.3, 1.04, 0.11, 5.0, 150, 250), (2, 163.0, 1.025, 0.085, 1.2, 600, 300),
       (3, 85.0, 1.025, 0.1225, 1.0, 335, 270)]
LOAD = [(5, 90, 30), (7, 100, 35), (9, 125, 50)]           # bus, P (MW), Q (MVAr)
DERATE, GROWTH = 0.85, 1.15
RADIAL = ["L1-4", "L3-6", "L8-2"]                           # generator connections


def network(dispatch=None, outage=None, cap9=0.0, rating_56=None):
    n = pypsa.Network()
    for b in range(1, 10):
        n.add("Bus", str(b), v_nom=V_N)
    for f, t, r, x, b, rate in BRANCH:                      # p.u. → ohm and siemens
        n.add("Line", f"L{f}-{t}", bus0=str(f), bus1=str(t), r=r * Z_B, x=x * Z_B, b=b / Z_B,
              s_nom=rate * DERATE)
    if rating_56:
        n.lines.loc["L5-6", "s_nom"] = rating_56
    for k, (b, p, vset, c2, c1, c0, pmax) in enumerate(GEN):
        n.buses.loc[str(b), "v_mag_pu_set"] = vset
        n.add("Generator", f"G{b}", bus=str(b), control="Slack" if k == 0 else "PV",
              p_nom=pmax, p_min_pu=10 / pmax, marginal_cost=c1, marginal_cost_quadratic=c2,
              p_set=np.nan if dispatch is None else dispatch[f"G{b}"])
    for b, p, q in LOAD:
        n.add("Load", f"D{b}", bus=str(b), p_set=p * GROWTH, q_set=q * GROWTH)
    if outage:
        n.lines.loc[outage, "active"] = False
    if cap9:
        n.add("ShuntImpedance", "cap9", bus="9", b=cap9 / V_N**2)
    return n


def loading(n):
    s = np.maximum(np.hypot(n.lines_t.p0, n.lines_t.q0), np.hypot(n.lines_t.p1, n.lines_t.q1)).iloc[0]
    return 100 * s / n.lines.s_nom


C0 = sum(g[5] for g in GEN)
opts = dict(solver_name="highs", solver_options={"output_flag": False})
# ------------------------------------------------------------ 1. economic dispatch (DC-OPF)
n = network(); n.optimize(**opts)                          # p_set = NaN: dispatch is free
ed = n.generators_t.p.iloc[0]
print("1. Economic dispatch (MW):", ed.round(1).to_dict(), "| cost %.0f $/h" % (n.objective + C0))
print("   LMP %.2f $/MWh at every bus" % n.buses_t.marginal_price.iloc[0].mean())
cost_ed = n.objective + C0

# ------------------------------------------------------------ 2. AC check, intact grid
m = network(ed); m.pf()
print("2. Intact grid: max loading %.1f %%, min |V| %.3f p.u." % (loading(m).max(), m.buses_t.v_mag_pu.
     iloc[0].min()))

# ------------------------------------------------------------ 3. N-1 screening (AC)
def n1(dispatch, **kw):
    rows = []
    for out in [l for l in network().lines.index if l not in RADIAL]:
        m = network(dispatch, outage=out, **kw); m.pf()
        L = loading(m).drop(out)
        rows.append({"outage": out, "worst line": L.idxmax(), "loading (%)": L.max(),
```

```
                      "min |V|": m.buses_t.v_mag_pu.iloc[0].min()})
    return pd.DataFrame(rows).set_index("outage").round(3)

print("3. N-1 screening of the economic dispatch\n", n1(ed).to_string())

# ------------------------------------------------------------ 4. security-constrained OPF
n = network()
n.optimize.optimize_security_constrained(branch_outages=[l for l in n.lines.index if l not in RADIAL],
    **opts)
sc = n.generators_t.p.iloc[0]
lmp = n.buses_t.marginal_price.iloc[0]
print("4. Secure dispatch (MW):", sc.round(1).to_dict(), "| extra cost %.1f $/h" % (n.objective + C0 -
    cost_ed))
print("   LMP %.2f-%.2f $/MWh" % (lmp.min(), lmp.max()))
print("   N-1 screening of the secure dispatch\n", n1(sc).to_string())

# ------------------------------------------------------------ 5. voltage support at bus 9
lo, hi = 0.0, 300.0                    # bisection on the post-outage voltage of bus 9
for _ in range(30):
    mid = (lo + hi) / 2
    m = network(sc, outage="L9-4", cap9=mid); m.pf()
    lo, hi = (mid, hi) if m.buses_t.v_mag_pu.iloc[0]["9"] < 0.95 else (lo, mid)
m = network(sc, outage="L9-4", cap9=hi); m.pf()
print("5. Loss of 9-4: %.0f MVAr at bus 9 restores %.3f p.u.; worst loading %.1f %%"
      % (hi, m.buses_t.v_mag_pu.iloc[0]["9"], loading(m).drop("L9-4").max()))

# ------------------------------------------------------------ 6. reinforcement option
print("6. Line 5-6 uprated to 200 MVA, economic dispatch: worst N-1 loading %.1f %%"
      % n1(ed, rating_56=200)["loading (%)"].max())
```

Step 6: results

Code 10: Output of Code 5.

```
1. Economic dispatch (MW): {'G1': 101.4, 'G2': 153.5, 'G3': 107.3} | cost 6429 $/h
   LMP 27.30 $/MWh at every bus
2. Intact grid: max loading 72.5 %, min |V| 0.984 p.u.
3. N-1 screening of the economic dispatch
        worst line  loading (%)  min |V|
outage
L4-5         L5-6       87.393    0.909
L5-6         L6-7       84.310    0.947
L6-7         L5-6       84.768    0.962
L7-8         L6-7       95.906    0.952
L8-9         L5-6      112.720    0.916
L9-4         L6-7       97.432    0.751
4. Secure dispatch (MW): {'G1': 119.8, 'G2': 142.7, 'G3': 99.8} | extra cost 54.1 $/h
   LMP 25.46-31.35 $/MWh
   N-1 screening of the secure dispatch
        worst line  loading (%)  min |V|
outage
L4-5         L5-6       87.418    0.908
L5-6         L6-7       78.365    0.950
L6-7         L5-6       78.941    0.962
L7-8         L6-7       95.921    0.951
L8-9         L5-6       98.656    0.920
L9-4         L6-7      106.100    0.747
5. Loss of 9-4: 62 MVAr at bus 9 restores 0.950 p.u.; worst loading 98.5 %
6. Line 5-6 uprated to 200 MVA, economic dispatch: worst N-1 loading 97.4 %
```

Step 7: verification

Generation $101.4 + 153.5 + 107.3 = 362.2$ MW $= 1.15 \times 315$ MW (DC model, no losses). ✓ The DC security constraint is re-checked with the full AC model: line 5-6 at 98.7 % after the loss of 8-9. ✓ The AC check also reveals what the DC model cannot see (next step).

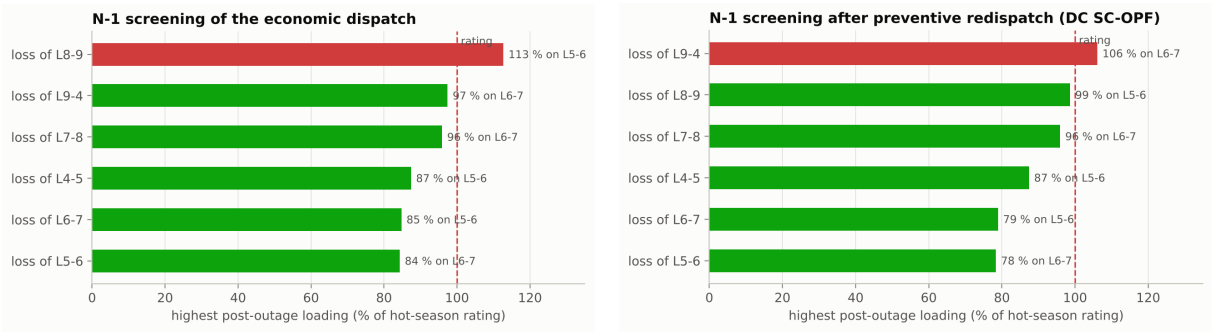


Figure 14: Worst post-outage loading: economic dispatch (left) and secure dispatch (right).

Step 8: interpretation

The cheapest dispatch is not N-1 secure: losing line 8-9 overloads line 5-6 to **113 %** (close to the 115 % of the statement, but computed). Preventive redispatch (18 MW moved from G2/G3 to G1) removes it for **54 $/h** (0.8 %); nodal prices separate (25.5 to 31.3 $/MWh). The AC check finds a second, hidden problem: losing line 9-4 drops bus 9 to 0.75 p.u. and overloads line 6-7 (106 %); **62 MVAr** of switched reactive support at bus 9 cures both. Uprating line 5-6 removes the flow problem but not the voltage problem.

What I say in front of a jury

"The IEEE 9-bus system shows that a line can exceed its capacity after an outage. I evaluated redispatch, reinforcement and reactive support. Redispatch costs less than 1 % and is the right first answer; the AC check proved that a voltage reserve at bus 9 is also needed."

8. TP case 4: IEEE 14-bus system, demand growth

Step 1: statement and question

Statement

Reference IEEE 14-bus system (14 nodes, generators, loads, transformers, lines). Demand grows: S0 = 100 %, S1 = 110 %, S2 = 120 %, S3 = 130 %. For each scenario compare: (1) minimum voltage, (2) number of buses in undervoltage, (3) maximum line loading, (4) active losses, (5) reactive power supplied by the generators. The statement gives an *illustrative* table (e.g. S3: 0.90 p.u., 112 %, 8.0 MW, 4 buses below 0.95).

Step 2: data and hypotheses

Official MATPOWER `case14.m` (dataset `data/networks/ieee14`); thermal ratings from the IEEE PES benchmark PGLib-OPF; generator reactive limits from MATPOWER (G1 0-10, G2 -40/50, G3 0/40, G6 and G8 -6/24 MVAr). All loads scale with the same factor; G2 follows, the slack takes the rest. Hypothesis: reactive limits **are enforced**.

Step 3: model

AC load flow plus reactive limits: a PV generator with $Q > Q_{max}$ becomes PQ at Q_{max} (PV $\rightarrow$ PQ switching); its bus voltage is then free. Beyond the maximum loadability the Jacobian is singular and no solution exists.

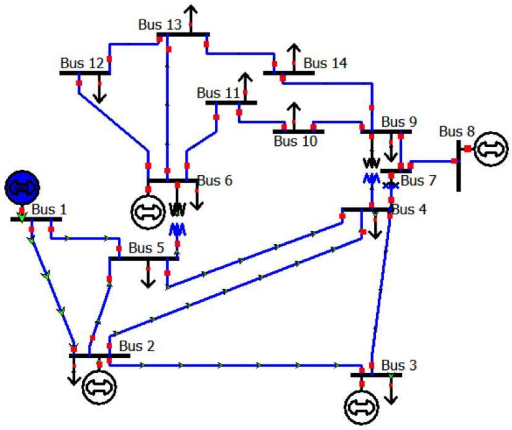


Figure 15: Original IEEE 14-bus figure of the practical work.

Step 4: method

Load flow with PV → PQ switching for S0 to S5; same scenarios without limits for comparison; increase the load in steps of 1 % until the load flow has no solution (maximum loadability).

Step 5: implementation

Code 11: TP case 4 in PyPSA.

```
"""TP case 4 - IEEE 14-bus system: demand growth S0-S3 (and beyond), with reactive limits.

The network is the PyPSA dataset data/networks/ieee14 (MATPOWER case14.m, PGLib ratings).
Generator reactive limits are stored in the custom columns q_min, q_max.
Run from the repository root:  python solutions/s06_tp4_demand_growth_ieee14.py
"""
import logging
import warnings

import numpy as np
import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
BASE = pypsa.Network("data/networks/ieee14")
print(BASE.generators[["bus", "control", "p_set", "q_min", "q_max"]].to_string())


def solve_with_q_limits(n, max_iter=10):
    """AC load flow with PV → PQ switching: a generator that hits a reactive limit loses
    its voltage control and is fixed at that limit; repeat until no limit is violated."""
    at_limit = []
    for _ in range(max_iter):
        if not n.pf()["converged"].all().all():         # no solution: beyond the nose point
            raise RuntimeError("load flow did not converge")
        q = n.generators_t.q.iloc[0]
        new = [(g, n.generators.at[g, "q_max"]) for g in n.generators.index[n.generators.control == "PV
    "]
                if q[g] > n.generators.at[g, "q_max"] + 1e-6]
        new += [(g, n.generators.at[g, "q_min"]) for g in n.generators.index[n.generators.control == "
    PV"]
                 if q[g] < n.generators.at[g, "q_min"] - 1e-6]
        if not new:
            return at_limit
        for g, q_lim in new:
            n.generators.loc[g, ["control", "q_set"]] = ["PQ", q_lim]
            at_limit.append(g)
    raise RuntimeError("reactive limits did not settle")

```

```python

def scenario(lam, q_limits=True):
    n = BASE.copy()
    n.loads[["p_set", "q_set"]] *= lam                  # every load grows by the same factor
    n.generators.loc["G2", "p_set"] *= lam              # G2 follows; the slack takes the rest
    at_limit = solve_with_q_limits(n) if q_limits else (n.pf() and [])
    v = n.buses_t.v_mag_pu.iloc[0]
    s = pd.concat([np.maximum(np.hypot(n.lines_t.p0, n.lines_t.q0), np.hypot(n.lines_t.p1, n.lines_t.q1)
    ).iloc[0] / n.lines.s_nom,
                   np.hypot(n.transformers_t.p0, n.transformers_t.q0).iloc[0] / n.transformers.s_nom])
    loss = (n.lines_t.p0 + n.lines_t.p1).iloc[0].sum() + (n.transformers_t.p0 + n.transformers_t.p1).
    iloc[0].sum()
    return {"min|V|": round(v.min(), 3), "bus": v.idxmin(), "n<0.95": int((v < 0.95).sum()),
            "max%": round(100 * s.max(), 1), "line": s.idxmax(), "loss MW": round(loss, 2),
            "Q MVAr": round(n.generators_t.q.iloc[0].sum(), 1), "at Q limit": ",".join(at_limit) or "-"}


rows = {f"S{k} ({int(100*l)} %)": scenario(l) for k, l in enumerate([1.0, 1.1, 1.2, 1.3, 1.4, 1.5])}
print("\nScenarios with reactive limits enforced")
print(pd.DataFrame(rows).T.round(3).to_string())
print("\nSame scenarios if reactive limits are ignored (PyPSA default): min |V| =")
print({k.split()[0]: scenario(l, False)["min|V|"] for k, l in zip(rows, [1.0, 1.1, 1.2, 1.3, 1.4, 1.5])
    })

lam = 1.0                                               # maximum loadability (step 0.01)
while True:
    try:
        r = scenario(round(lam + 0.01, 2))
        if r["min |V|"] < 0.5:
            break
        lam = round(lam + 0.01, 2)
    except Exception:
        break
print(f"\nMaximum loadability with reactive limits: lambda = {lam:.2f} "
      f"({BASE.loads.p_set.sum() * lam:.0f} MW, margin +{100 * (lam - 1):.0f} %)")
```

Step 6: results

Code 12: Output of Code 6.

```
     bus control  p_set  q_min  q_max
name
G1     1   Slack  232.4    0.0   10.0
G2     2      PV   40.0  -40.0   50.0
G3     3      PV    0.0    0.0   40.0
G6     6      PV    0.0   -6.0   24.0
G8     8      PV    0.0   -6.0   24.0

Scenarios with reactive limits enforced
           min|V| bus n<0.95  max%  line loss MW Q MVAr   at Q limit
S0 (100 %)   1.01   3      0  59.1  L1-5   13.39   82.4            -
S1 (110 %)   1.01   3      0  65.3  L1-5    16.4  102.1           G2
S2 (120 %)  1.006   3      0  71.7  L1-5   19.78  123.5     G2,G3,G6
S3 (130 %)  0.983   3      0  78.7  L1-5   23.86  149.4  G2,G3,G6,G8
S4 (140 %)  0.954  14      0  86.6  L1-5    28.9  179.4  G2,G3,G6,G8
S5 (150 %)  0.909  14      9  95.5  L1-5    35.2  215.1  G2,G3,G6,G8

Same scenarios if reactive limits are ignored (PyPSA default): min |V| =
{'S0': 1.01, 'S1': 1.01, 'S2': 1.01, 'S3': 1.008, 'S4': 1.005, 'S5': 1.001}

Maximum loadability with reactive limits: lambda = 1.00 (259 MW, margin +0 %)
```

Step 7: verification

S0 reproduces the published IEEE 14-bus solution (slack 232.4 MW, losses 13.393 MW) and our own Newton-Raphson solver to 10^{-13}. ✓ Losses grow faster than demand ($\times 2.6$ for $\times 1.5$): consistent with I^2R. ✓

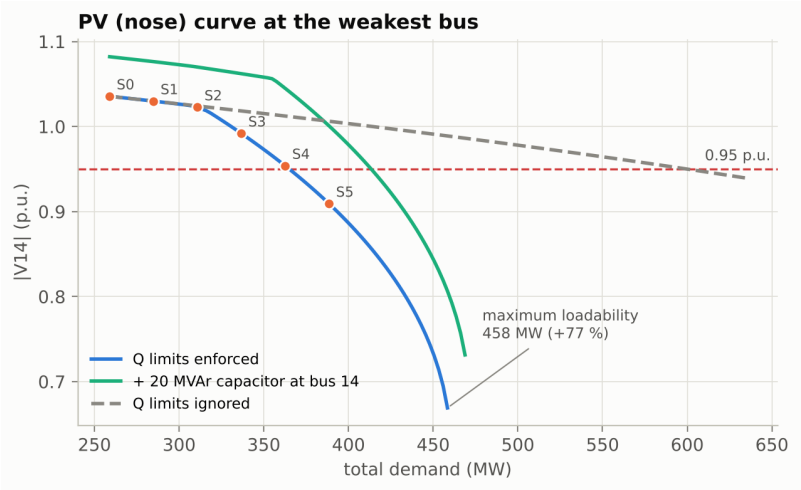


Figure 16: PV (nose) curve at bus 14: S0 to S5 and the maximum loadability.

Step 8: interpretation

Up to +30 % the indicators look healthy (min 0.983 p.u., no line above 79 %), yet the four voltage-control units are all at their reactive limit: **the reactive reserve runs out first**. At +50 % nine buses fall below 0.95 p.u. The maximum loadability is +77 %; ignoring reactive limits (the default behaviour) would wrongly suggest more than +145 %.

What I say in front of a jury

"I studied the sensitivity of the IEEE 14-bus network to demand growth. Voltages decrease, losses and line loadings increase, and the reactive reserve is exhausted at +30 %. Beyond about +40 % operating limits are violated: reactive compensation and reinforcement must be planned before that point."

9. TP case 5: solar plant in the 14-bus network, reverse flow and storage

Step 1: statement and question

Statement

Scenarios: S0 no solar; S1 50 MW of solar; S2 100 MW of solar; S3 100 MW of solar + storage. Solar integration can reverse flows, raise voltages locally, reduce conventional output and change reactive needs. *Example: a substation receives 40 MW; a local plant produces 60 MW while the local load stays at 40 MW: the surplus* $60 - 40 = 20$ *MW flows back upstream.* A battery does not automatically solve voltage or congestion problems: its location, power, energy and control strategy must be studied.

Step 2: data and hypotheses

IEEE 14-bus (case 4) with the plant at **bus 14**, the weakest node (local load about 14 MW at noon). Real solar profile of 16 March 2023 (clearest day of the first quarter, Niamey). Battery 50 MW / 200 MWh at bus 14. Voltage limits 0.94-1.06 p.u. on the customer buses (generator buses are held at their set points; bus 7 is a transformer star point).



Figure 17: Original illustration: solar plant with battery storage.

Step 3: model

Reverse flow: $P_{surplus} = P_{PV} - P_{load}$ leaves the node. Voltage rise: $\Delta V \approx (RP + XQ)/V$ with P now exported; absorbing reactive power ($Q < 0$, PF 0.95: $Q = -0.33P$) compensates it. Hosting capacity: the largest PV power before the first limit is violated.

Step 4: method

(1) Hosting capacity at noon for three strategies (1 MW steps). (2) Flows into bus 14 versus PV size. (3) The four scenarios of the statement plus S4 = S3 + reactive control + lower set points, over 24 hours. Battery rule: peak shaving (store the PV above the lowest threshold that fills 200 MWh; return it 18:00-22:00).

Step 5: implementation

Code 13: TP case 5 in PyPSA.

```
"""TP case 5 - Solar plant at bus 14 of the IEEE 14-bus system: hosting capacity, reverse
power flow, battery storage. Real solar profile of 16 March 2023 (Niamey).

Dataset data/networks/ieee14_pv_day: IEEE 14 + PV plant 'pv14' + battery 'battery14' at
bus 14, 24 hourly snapshots. Run:  python solutions/s07_tp5_solar_storage_ieee14.py
"""
import logging
import warnings

import numpy as np
import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
DAY = pypsa.Network("data/networks/ieee14_pv_day")
CF = DAY.generators_t.p_max_pu["pv14"]                  # real hourly capacity factor
NOON = CF.idxmax()
MONITORED = ["4", "5", "9", "10", "11", "12", "13", "14"]  # customer buses + PV bus
V_MIN, V_MAX, TAN = 0.94, 1.06, np.tan(np.arccos(0.95))


def q_limited_pf(n):
    """AC load flow with generator reactive limits (PV → PQ switching), first snapshot."""
    for _ in range(10):
        if not n.pf(snapshots=n.snapshots[:1])["converged"].all().all():
            raise RuntimeError("no load-flow solution")
        q = n.generators_t.q.iloc[0]
        pv = n.generators.index[(n.generators.control == "PV") & n.generators.q_max.notna()]
        hit = [g for g in pv if q[g] > n.generators.at[g, "q_max"] + 1e-6 or q[g] < n.generators.at[g,
 "q_min"] - 1e-6]
        if not hit:
            return
        for g in hit:
            lim = "q_max" if q[g] > n.generators.at[g, "q_max"] else "q_min"
```

```
            n.generators.loc[g, ["control", "q_set"]] = ["PQ", n.generators.at[g, lim]]


def lower_set_points(n, dv):
    buses = n.generators.bus[n.generators.control.isin(["PV", "Slack"])]
    n.buses.loc[buses, "v_mag_pu_set"] -= dv


def at_noon(pv_mw, absorb=False, dv=0.0):
    n = DAY.copy()
    n.set_snapshots([NOON])
    lower_set_points(n, dv)
    n.generators_t.p_set.loc[:, "pv14"] = pv_mw
    n.generators_t.q_set.loc[:, "pv14"] = -TAN * pv_mw if absorb else 0.0   # PF 0.95 absorbing
    n.storage_units_t.p_set.loc[:, "battery14"] = 0.0
    q_limited_pf(n)
    return n


def first_violation(n):
    v = n.buses_t.v_mag_pu.iloc[0][MONITORED]
    s = pd.concat([np.hypot(n.lines_t.p0, n.lines_t.q0).iloc[0] / n.lines.s_nom,
                   np.hypot(n.transformers_t.p0, n.transformers_t.q0).iloc[0] / n.transformers.s_nom])
    if v.max() > V_MAX + 1e-9:
        return "overvoltage"
    if v.min() < V_MIN - 1e-9:
        return "undervoltage"
    return "thermal" if s.max() > 1 else None


# ------------------------------------------------------------ 1. hosting capacity at noon
print("1. Hosting capacity of bus 14 at noon (1 MW steps)")
for name, absorb, dv in [("unity PF, original set points", False, 0.0),
                         ("unity PF, set points -0.03 p.u.", False, 0.03),
                         ("PF 0.95 absorbing, set points -0.03", True, 0.03)]:
    for p in range(0, 251):
        try:
            why = first_violation(at_noon(p, absorb, dv))
        except RuntimeError:
            why = "no solution"
        if why:
            print(f"   {name:38s}: {p - 1:4d} MW (limited by {why})")
            break

# ------------------------------------------------------------ 2. reverse power flow
print("\n2. Flow into bus 14 at noon (original set points, MW)")
for p in [0, 10, 14, 20, 50, 100]:
    n = at_noon(p)
    flow = n.lines_t.p0.iloc[0][["L9-14", "L13-14"]]
    print(f"   PV {p:3d} MW: 9→14 {flow['L9-14']:7.2f}, 13→14 {flow['L13-14']:7.2f}, total {flow.sum()
     :7.2f}")

# ------------------------------------------------------------ 3. the five scenarios over 24 h
def battery(pv, p_max=50.0, e_max=200.0, eta=0.95):
    """Peak shaving: store the PV above the lowest threshold whose surplus fits in e_max,
    return the energy evenly from 18:00 to 22:00. Sign: + discharge, - charge (PyPSA)."""
    lo, hi = 0.0, float(pv.max())
    for _ in range(50):
        t = (lo + hi) / 2
        lo, hi = (t, hi) if (pv - t).clip(0, p_max).sum() * eta > e_max else (lo, t)
    charge = (pv - hi).clip(0, p_max)
    evening = (pv.index.hour >= 18) & (pv.index.hour < 22)
    out = -charge
    out[evening] = min(p_max, charge.sum() * eta * eta / evening.sum())
    return out


def day(pv_mw, bat=False, absorb=False, dv=0.0):
    n = DAY.copy()
    lower_set_points(n, dv)
    pv = pv_mw * CF
    n.generators_t.p_set["pv14"] = pv
    n.generators_t.q_set["pv14"] = -TAN * pv if absorb else 0 * pv
```

```
    n.storage_units_t.p_set["battery14"] = battery(pv) if bat else 0 * pv
    n.pf()
    vmon = n.buses_t.v_mag_pu[MONITORED].max(axis=1)
    inflow = n.lines_t.p0["L9-14"] + n.lines_t.p0["L13-14"]
    return {"max V14": n.buses_t.v_mag_pu["14"].max(), "overV h": int((vmon > V_MAX + 1e-9).sum()),
            "reverse h": int((inflow < 0).sum()), "export MW": max(0.0, -inflow.min()),
            "loss MWh": ((n.lines_t.p0 + n.lines_t.p1).sum(axis=1)
                         + (n.transformers_t.p0 + n.transformers_t.p1).sum(axis=1)).sum()}

rows = {"S0 no PV": day(0), "S1 PV 50": day(50), "S2 PV 100": day(100),
        "S3 PV 100 + battery": day(100, bat=True),
        "S4 S3 + PF0.95 + setpts": day(100, bat=True, absorb=True, dv=0.03)}
print("\n3. Scenarios over 16 March 2023")
print(pd.DataFrame(rows).T.round(3).to_string())
```

Step 6: results

Code 14: Output of Code 7.

```
1. Hosting capacity of bus 14 at noon (1 MW steps)
   unity PF, original set points         :    3 MW (limited by overvoltage)
   unity PF, set points -0.03 p.u.       :   66 MW (limited by overvoltage)
   PF 0.95 absorbing, set points -0.03   :  129 MW (limited by undervoltage)

2. Flow into bus 14 at noon (original set points, MW)
   PV   0 MW: 9->14    8.79, 13->14    5.13, total   13.92
   PV  10 MW: 9->14    2.69, 13->14    1.11, total    3.80
   PV  14 MW: 9->14    0.27, 13->14   -0.48, total   -0.21
   PV  20 MW: 9->14   -3.33, 13->14   -2.86, total   -6.19
   PV  50 MW: 9->14  -20.96, 13->14  -14.43, total  -35.39
   PV 100 MW: 9->14  -48.96, 13->14  -32.83, total  -81.79

3. Scenarios over 16 March 2023
                         max V14  overV h  reverse h  export MW  loss MWh
S0 no PV                   1.057     15.0        0.0      0.000   210.963
S1 PV 50                   1.073     20.0        9.0     25.178   186.198
S2 PV 100                  1.098     20.0       10.0     62.416   180.229
S3 PV 100 + battery        1.080     24.0       14.0     37.512   166.579
S4 S3 + PF0.95 + setpts    1.046      0.0       14.0     37.436   183.759
```

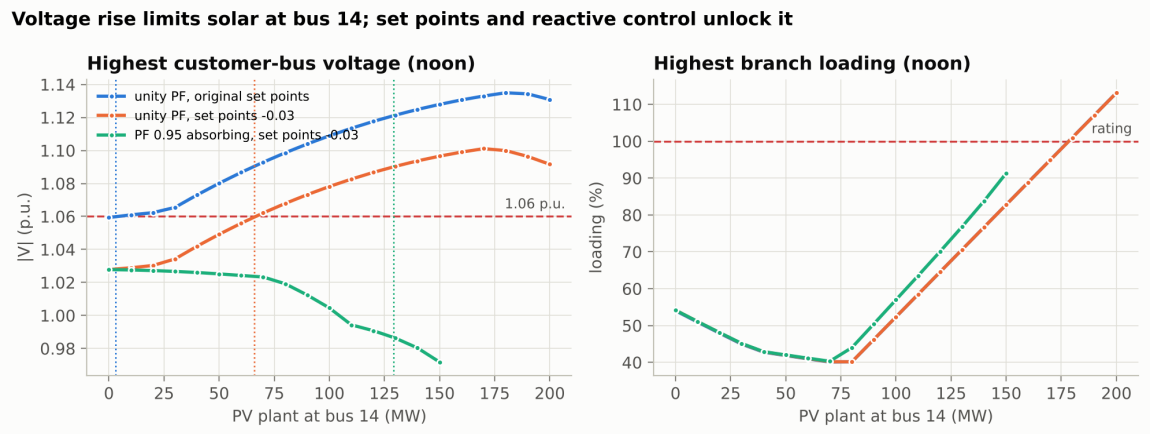


Figure 18: Hosting capacity of bus 14 at noon: voltage, not line capacity, is the limit.

Step 7: verification

Reverse flow appears between 10 and 14 MW of PV: exactly where PV equals the local noon load (13.8 MW), as the statement's formula $P_{surplus} = P_{PV} - P_{load}$ predicts. ✓ With 100 MW the export is 81.8 MW at noon $\approx 100 - 13.8 -$ losses. ✓

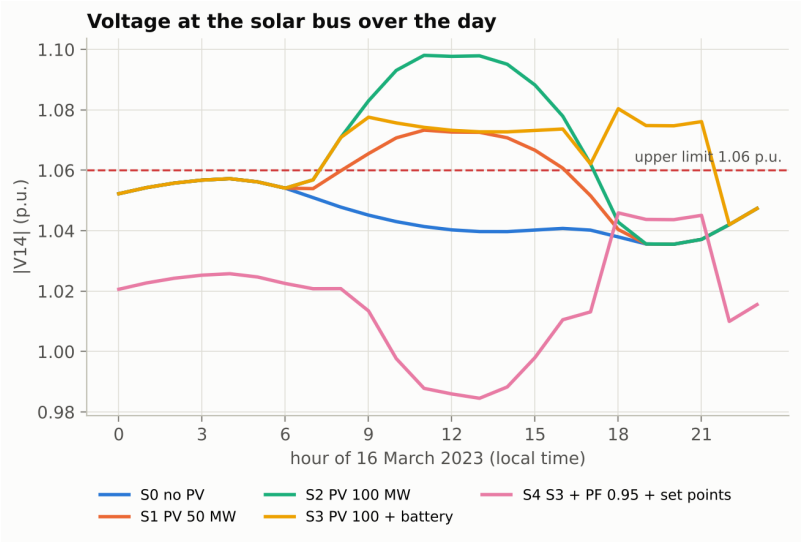


Figure 19: Voltage at the solar bus over the day for the five scenarios.

Step 8: interpretation

The original voltage set points leave almost no margin (hosting capacity 3 MW). Lowering the set points by 0.03 p.u. (free) gives 66 MW; adding inverter reactive control gives 129 MW. The battery reduces the peak voltage (1.098 → 1.080 p.u.) and the export (62 → 37 MW), but its evening discharge at the weak node raises the voltage there: **storage needs a voltage-aware strategy**. The full package (S4) removes every overvoltage hour.

What I say in front of a jury

"Solar changes where power is injected. When local production exceeds local demand the flow reverses, and the voltage rises. I measured the hosting capacity and showed that set points and inverter reactive control increase it from 3 to 129 MW; the battery helps only with the right location and control."

10. Real case A: solar and storage for the Niamey area, every hour of 2023

Step 1: statement and question

Statement (decision case)

The Niamey area is supplied by a thermal plant (HFO/diesel, fuel 180 $/MWh) and by imports (70 $/MWh), which can be interrupted (as in 2023). How much solar and battery capacity minimises the yearly cost of supply, with and without imports? What does it change for fuel, CO_2 and the grid? Six scenarios ("islanded" = imports interrupted): A imports, no PV; B no imports, no PV; C no imports, 100 MW of PV; D0 no imports, optimal PV only; D no imports, optimal PV + battery; E imports, optimal PV + battery.

Table 5: Real case A: data (dataset `data/networks/niamey_grid_2023`).

Item	Value	Source / status
Weather	hourly 2023, irradiance and temperature, Niamey	NASA POWER (real)
PV output	pvlib: Hay-Davies, Faiman, PVWatts, 10 % losses; 1 710 kWh/kWp	same chain as Renewables.ninja
Demand	726 GWh, peak 105 MW, temperature-driven	model (assumption)
Grid	five-bus network of TP case 2 + 15 MVAr bank	fictitious
PV cost	800 \$/kW, 25 years, 8 % ⇒ 86.9 k\$/MW/yr	IRENA (2024) range
Battery	4 h, 1 150 \$/kW, 15 years ⇒ 157.4 k\$/MW/yr; 90 % round trip	NREL ATB range
Islanded operation	thermal must-run 20 MW (inertia, frequency control)	assumption

Step 2: data and hypotheses

Step 3: model

Linear optimal power flow with investment over the 8760 hours:

$$\min \sum_{a \in \{PV,\, bat\}} c_a^{cap} \bar{P}_a + \sum_t \sum_g c_g \, p_{g,t}$$

subject to nodal balance every hour, $|P_{\ell,t}| \le 0.9\, s_{nom}$, $p_{PV,t} \le \bar{P}_{PV}\, CF_t$, battery state of charge $e_t = e_{t-1} + 0.95\, p_t^{ch} - p_t^{dis}/0.95$, $0 \le e_t \le 4\bar{P}_{bat}$, and the must-run when islanded.

Step 4: method

Solve the six scenarios with `n.optimize()` (HiGHS); then take the optimal hourly dispatch of D and run the full AC load flow at the two critical hours (evening peak, solar peak).

Step 5: implementation

Code 15: Real case A in PyPSA.

```python
"""Real case A - Niamey area: least-cost solar and storage, every hour of 2023.

Dataset data/networks/niamey_grid_2023: five-bus grid of TP case 2 (with the 15 MVAr bank),
8760 hourly snapshots; real NASA POWER weather (PV capacity factor in generators_t.p_max_pu),
temperature-driven demand (loads_t.p_set). PV and battery are extendable (capital costs set
from src/pscs/assumptions.py). Run:  python solutions/s08_real_case_niamey.py
"""
import logging
import warnings

import numpy as np
import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
PATH = "data/networks/niamey_grid_2023"
FUEL, IMPORT, CO2 = 180.0, 70.0, 0.70          # $/MWh, $/MWh, tCO2/MWh
MUST_RUN = 20.0                                # MW of thermal online when islanded

n0 = pypsa.Network(PATH)
print(n0.generators[["bus", "p_nom", "p_nom_extendable", "marginal_cost", "capital_cost"]].round(0).
    to_string())
print(n0.storage_units[["bus", "max_hours", "capital_cost", "efficiency_store"]].round(2).to_string())
demand = n0.loads_t.p_set.sum(axis=1)
cf = n0.generators_t.p_max_pu["pv"]
print(f"demand {demand.sum()/1e3:.0f} GWh, peak {demand.max():.0f} MW; PV capacity factor {cf.mean()
    :.1%}")


```

```python
def build(imports, pv=None, battery=None):
    """pv / battery = None → optimised by PyPSA; a number → fixed capacity (MW)."""
    n = pypsa.Network(PATH)
    if not imports:
        n.generators.loc["import", "p_nom"] = 0.0
        n.generators.loc["thermal", "p_min_pu"] = MUST_RUN / n.generators.at["thermal", "p_nom"]
    if pv is not None:
        n.generators.loc["pv", ["p_nom_extendable", "p_nom"]] = [False, pv]
    if battery is not None:
        n.storage_units.loc["battery", ["p_nom_extendable", "p_nom"]] = [False, battery]
    return n


SCEN = {"A imports": (True, 0, 0), "B islanded": (False, 0, 0),           # islanded = no imports
        "C islanded+100PV": (False, 100, 0), "D0 islanded+optPV": (False, None, 0),
        "D islanded+opt": (False, None, None), "E imports+opt": (True, None, None)}
rows, nets = {}, {}
for name, args in SCEN.items():
    n = build(*args)
    n.optimize(solver_name="highs", solver_options={"output_flag": False})   # linear OPF, 8760 h
    p = n.generators_t.p
    pv_mw, bat_mw = n.generators.p_nom_opt["pv"], n.storage_units.p_nom_opt["battery"]
    capex = pv_mw * n.generators.at["pv", "capital_cost"] + bat_mw * n.storage_units.at["battery", "
     capital_cost"]
    opex = (p["thermal"] * FUEL + p["import"] * IMPORT).sum()
    avail = (pv_mw * cf).sum()
    rows[name] = {"PV MW": pv_mw, "bat MW": bat_mw, "bat MWh": 4 * bat_mw,
                  "solar %": 100 * p["pv"].sum() / demand.sum(),
                  "curtail %": 100 * (1 - p["pv"].sum() / avail) if avail else 0,
                  "CO2 kt": p["thermal"].sum() * CO2 / 1e3, "cost M$": (capex + opex) / 1e6,
                  "$/MWh": (capex + opex) / demand.sum(),
                  "max line %": 100 * (n.lines_t.p0.abs() / n.lines.s_nom).max().max()}
    nets[name] = n
print(pd.DataFrame(rows).T.round(1).to_string())

# ------------------------------------------------------------ AC check of scenario D
nD = nets["D islanded+opt"]
nD.model.solver_model = None                          # needed before copying a solved network
hours = pd.DatetimeIndex([demand.idxmax(), nD.generators_t.p["pv"].idxmax()])
m = nD.copy()
m.set_snapshots(hours)
m.generators_t.p_set = nD.generators_t.p.loc[hours]          # optimal dispatch → set points
m.storage_units_t.p_set = nD.storage_units_t.p.loc[hours]
m.generators["control"] = "PQ"
m.generators.loc["thermal", "control"] = "Slack"             # the slack covers the AC losses
m.pf()
ac = pd.DataFrame({"min |V|": m.buses_t.v_mag_pu.min(axis=1), "max |V|": m.buses_t.v_mag_pu.max(axis=1),

                   "max loading %": 100 * (np.hypot(m.lines_t.p0, m.lines_t.q0) / m.lines.s_nom).max(
    axis=1),
                   "solar MW": m.generators_t.p["pv"]}, index=hours)
print("\nAC load flow of scenario D at the evening peak and at the solar peak\n", ac.round(3).to_string
    ())
```

Step 6: results

Code 16: Output of Code 8.

```
        bus  p_nom  p_nom_extendable  marginal_cost  capital_cost
name
thermal   1  200.0             False          180.0           0.0
import    1   80.0             False           70.0           0.0
pv        3    0.0              True            0.0       86943.0
        bus  max_hours  capital_cost  efficiency_store
name
battery   3        4.0     157353.98              0.95
demand 726 GWh, peak 105 MW; PV capacity factor 19.5%
                 PV MW  bat MW  bat MWh  solar %  curtail %  CO2 kt  cost M$  $/MWh  max line %
A imports          0.0     0.0      0.0      0.0        0.0    35.2     56.4   77.6        55.4
B islanded         0.0     0.0      0.0      0.0        0.0   508.2    130.7  180.0        55.4
C islanded+100PV   100.0   0.0      0.0      23.0       2.3   391.3    109.3  150.5        62.5
```

```
D0 islanded+optPV  153.9     0.0      0.0     29.0       20.0   360.9    106.2  146.2        63.8
D islanded+opt     333.5   195.7    782.9     71.5        8.9   163.0    101.7  140.1        67.3
E imports+opt      143.9     8.3     33.4     33.0        2.6     6.8     49.0   67.5        67.1

AC load flow of scenario D at the evening peak and at the solar peak
                     min |V|  max |V|  max loading %  solar MW
2023-05-04 19:00:00    0.968     1.02         69.291    -0.000
2023-04-16 13:00:00    0.972     1.02         64.263   273.442
```

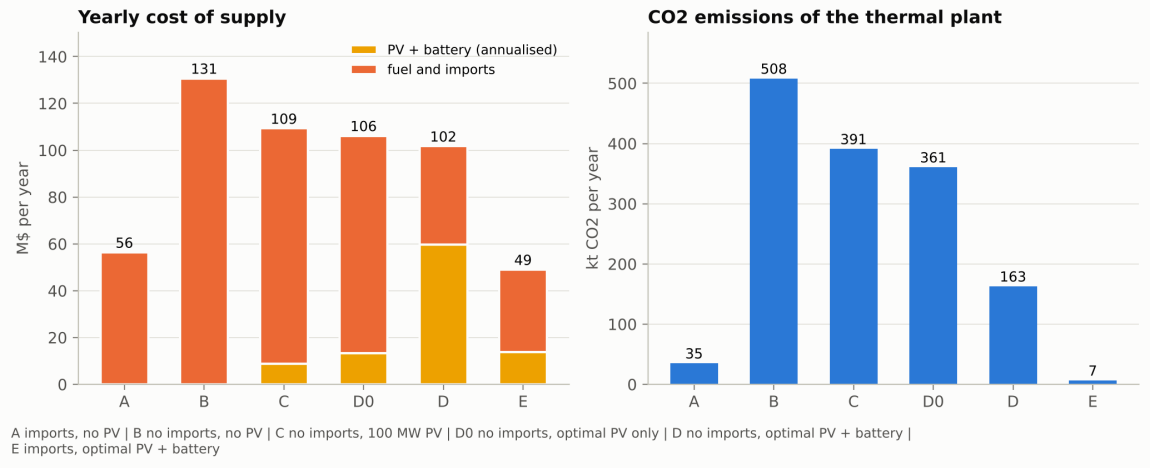


Figure 20: Yearly cost of supply and CO_2 emissions of the six scenarios.

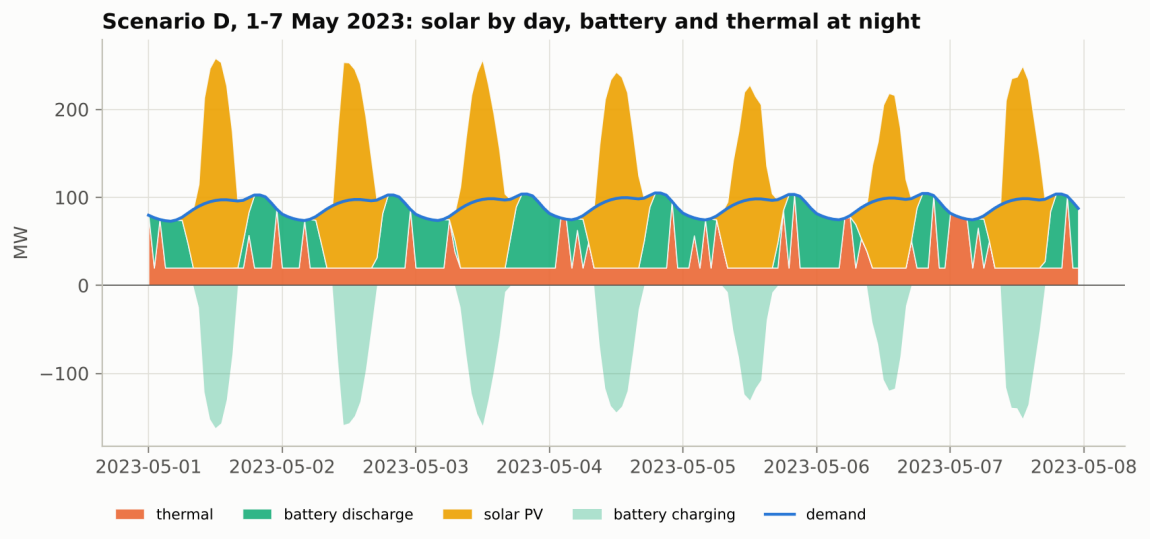


Figure 21: Scenario D, 1-7 May 2023: solar by day, battery in the evening, thermal must-run at night.

Step 7: verification

Solar LCOE if fully used: $86\,943/1\,710 = 51$ \$/MWh < 70 and 180 \$/MWh: PV must be built in every optimised scenario. ✓ Scenario B: 726 GWh $\times$ 180 \$/MWh = 130.7 M\$ (same as the output). ✓ The AC load flow of scenario D stays within 0.968-1.02 p.u. and 69 % loading. ✓

Step 8: interpretation

Without storage the optimum stops at about 154 MW of PV (20 % curtailed): midday demand minus the must-run cannot absorb more. **Storage, not the grid, unlocks solar**: with a 196 MW / 783 MWh battery, 333 MW of PV supply 72 % of the demand; fuel -68 %, cost -22 % (180 $\rightarrow$ 140 \$/MWh). With imports,

adding about 145 MW of PV still lowers the cost from 78 to 68 $/MWh. The thermal plant stays for the monsoon weeks.

What I say in front of a jury

"With real hourly weather for 2023, I optimised investment and operation together. When imports stop, solar plus a four-hour battery is the least-cost answer and cuts fuel by two thirds; the AC check confirms the plan is feasible."

11. Real case B: off-grid mini-grid in the Agadez region

Step 1: statement and question

Statement (decision case)

An isolated town (peak 2 MW) is supplied by diesel generators, with fuel trucked over long distances. What mix of PV and batteries minimises the cost of electricity, how much diesel does it save, and how robust is the answer to the fuel price (0.8 to 1.4 $/litre)?

Step 2: data and hypotheses

Real hourly weather 2023 for Agadez (NASA POWER; PV 1 857 kWh/kWp); demand 12.4 GWh/yr (temperature-driven model); existing 2.5 MW genset, 0.28 l/kWh, 20 $/MWh O&M, 0.74 tCO_2/MWh; remote-site PV 1 000 $/kW, battery 350 $/kWh + 200 $/kW.

Step 3: model

Same capacity-expansion model as case A on a single bus: diesel cost $c_d = 1000 \cdot \text{price} \cdot 0.28 + 20$ $/MWh; cost of electricity = (annualised capital + fuel) / energy.

Step 4: method

Diesel-only reference, optimal hybrid at 1.00 $/l, then the same optimisation for four fuel prices.

Step 5: implementation

Code 17: Real case B in PyPSA.

```python
"""Real case B - Off-grid mini-grid in the Agadez region: least-cost PV + battery + diesel.

Dataset data/networks/agadez_minigrid_2023: one bus, 8760 hourly snapshots of 2023, real
NASA POWER weather, demand peak 2 MW, existing 2.5 MW diesel genset; PV and a 4-hour battery
are extendable. Run:  python solutions/s09_real_case_agadez.py
"""
import logging
import warnings

import pandas as pd
import pypsa

warnings.filterwarnings("ignore"); logging.disable(logging.CRITICAL)
PATH = "data/networks/agadez_minigrid_2023"
SFC, VOM, CO2 = 0.28, 20.0, 0.74                  # l/kWh, $/MWh, tCO2/MWh of diesel


def solve(diesel_price=1.0, hybrid=True):
    n = pypsa.Network(PATH)
    n.generators.loc["diesel", "marginal_cost"] = diesel_price * SFC * 1000 + VOM   # $/MWh
    if not hybrid:
```

```
        n.generators.loc["pv", ["p_nom_extendable", "p_nom"]] = [False, 0]
        n.storage_units.loc["battery", ["p_nom_extendable", "p_nom"]] = [False, 0]
    n.optimize(solver_name="highs", solver_options={"output_flag": False})
    E = n.loads_t.p_set.sum().sum()
    d = n.generators_t.p["diesel"]
    capex = (n.generators.p_nom_opt["pv"] * n.generators.at["pv", "capital_cost"]
             + n.storage_units.p_nom_opt["battery"] * n.storage_units.at["battery", "capital_cost"])
    fuel = (d * n.generators.at["diesel", "marginal_cost"]).sum()
    return {"PV MW": n.generators.p_nom_opt["pv"], "battery MWh": 4 * n.storage_units.p_nom_opt["
     battery"],
            "renewable %": 100 * (1 - d.sum() / E), "diesel Ml": d.sum() * SFC / 1e3,
            "CO2 t": d.sum() * CO2, "$/kWh": (capex + fuel) / E / 1000}

print(pd.DataFrame({"diesel only": solve(hybrid=False), "optimal hybrid": solve()}).T.round(3).
    to_string())
print("\nSensitivity to the diesel price ($/litre)")
print(pd.DataFrame({p: solve(p) | {"$/kWh diesel only": solve(p, False)["$/kWh"]} for p in [0.8, 1.0,
    1.2, 1.4]}).T.round(3).to_string())
```

Step 6: results

Code 18: Output of Code 9.

```
                PV MW  battery MWh  renewable %  diesel Ml     CO2 t  $/kWh
diesel only     0.000        0.000        0.000      3.485  9209.455  0.300
optimal hybrid  7.508       17.931       93.812      0.216   569.891  0.163

Sensitivity to the diesel price ($/litre)
     PV MW  battery MWh  renewable %  diesel Ml    CO2 t  $/kWh  $/kWh diesel only
0.8  7.233       17.479       92.173      0.273  720.863  0.159             0.244
1.0  7.508       17.931       93.812      0.216  569.891  0.163             0.300
1.2  7.718       18.495       95.116      0.170  449.817  0.166             0.356
1.4  7.924       19.107       96.298      0.129  340.963  0.168             0.412
```

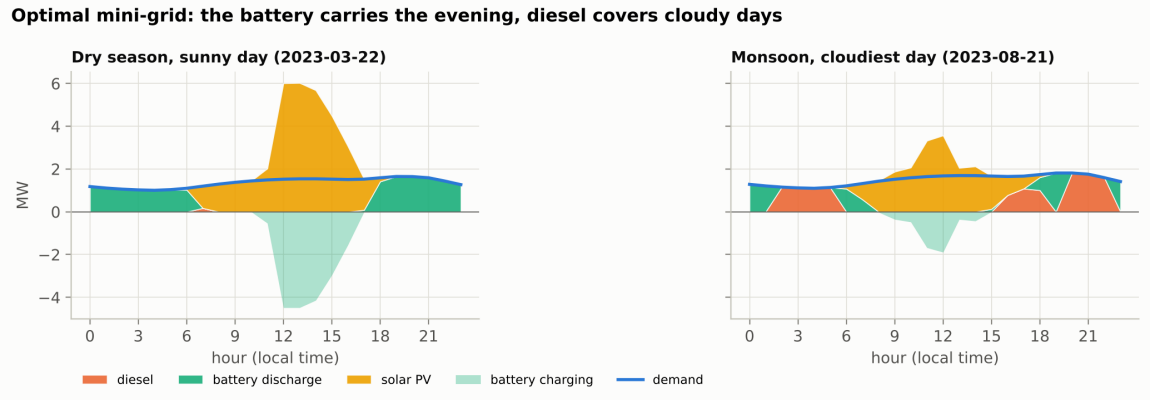


Figure 22: Optimal mini-grid: a sunny dry-season day and the cloudiest monsoon day.

Step 7: verification

Diesel only: $0.28 \times 1000 + 20 = 300$ \$/MWh $= 0.30$ \$/kWh (no capital cost: existing genset). ✓ Litres: 12 445 MWh $\times$ 280 l/MWh = 3.48 million litres. ✓

Step 8: interpretation

The hybrid (7.5 MW PV, 18 MWh battery) supplies 94 % of the energy, cuts diesel by 94 % and the cost from 0.30 to 0.16 \$/kWh; it stays cheaper at every fuel price tested. The genset remains as back-up for cloudy monsoon days.

What I say in front of a jury

"For an isolated town, solar plus storage halves the cost of electricity and removes almost all diesel, whatever the fuel price. The investment replaces a volatile fuel bill by a predictable cost."

12. The cases side by side: what changes

Table 6: Problem, cause, remedy and key number of each case.

Case	Problem	Physical cause	First remedy	Key number
Ex. 5	state of the grid	non-linear equations	Newton-Raphson	4 iterations, $\lvert U_3\rvert = 0.951$
TP 1	power sharing	slack closes the balance	solar reduces thermal	50.3 $\rightarrow$ 20.2 MW
TP 2	undervoltage	long line, reactive load ($X \approx 3R$)	local capacitor bank	0.919 $\rightarrow$ 0.959 p.u.
TP 3	congestion, N-1	outage redistributes flows	preventive redispatch	113 $\rightarrow$ 99 %, 54 \$/h
TP 4	demand growth	reactive reserve exhausted	reactive compensation	limit +77 %
TP 5	reverse flow	export raises voltage	set points, inverter Q	3 $\rightarrow$ 129 MW
Case A	planning	midday surplus, evening peak	storage	cost $-22\,\%$, fuel $-68\,\%$
Case B	energy access	expensive diesel	PV + battery	0.30 $\rightarrow$ 0.16 \$/kWh

13. Secrets and pitfalls

Secret n°1: always check the data before computing

A load flow that does not converge often has **no solution**. In Exercise 5 the literal line data cannot carry the load: swap the values, state the hypothesis, and compare with the other reading.

Secret n°2: reactive power moves voltage, active power moves angles

On transmission lines $X \gg R$: undervoltage is cured by local reactive power (TP 2), overvoltage by absorbing it (TP 5).

Secret n°3: enforce generator reactive limits

PyPSA's `n.pf()` holds PV voltages whatever reactive power it takes. Without PV $\rightarrow$ PQ switching, TP 4 would claim +145 % of loadability instead of +77 %.

Secret n°4: a linear optimisation must be checked with the AC load flow

The DC security-constrained OPF of TP 3 fixed the flows but could not see the voltage collapse at bus 9.

Secret n°5: PyPSA details that cost hours

A numeric `p_set` **fixes** a generator in `n.optimize()` (set it to NaN); the slack needs a numeric `p_set` (0) for `n.pf()`; lines are in Ω/S, transformers in p.u. on their own rating; a solved network is copied only after `n.model.solver_model = None`.

Secret n°6: quick plausibility checks

Active balance (generation = load + losses); $\Delta V \approx (RP + XQ)/V$ on the critical line; losses of a few percent; voltages 0.9-1.1 p.u.; a result outside these ranges signals a unit or data error.

14. Reflex sheet (to copy at the start of an exam or a study)

Check-list

1. Read the statement; write the question in one sentence; copy the diagram and mark bus types (slack, PV, PQ).
2. List the data with units; convert to PyPSA units ($Z_B = \text{kV}^2/\text{MVA}$); write every hypothesis.
3. Write the model: unknowns, equations, limits (and costs if it is an optimisation).
4. Choose the method: `n.pf()`, Q-limit loop, `n.optimize()`, security-constrained OPF, bisection on a limit.
5. Code it; print voltages, flows, loadings, losses.
6. Verify: balances, hand estimate, independent solver, orders of magnitude.
7. Interpret: which bus or line, why (physics), what to do (cheapest effective remedy first).
8. Present in five steps: context, problem, method, results, recommendations.